



Universität Hamburg

DER FORSCHUNG | DER LEHRE | DER BILDUNG

MASTER THESIS

Development of a dynamic layout creation algorithm for process mining process models: Procedure model, prototyping and evaluation

Author:

Mats Schrader

Matriculation Number: 6837119

MIN Faculty

Department of Computer Science

Course: MSc Informatics

First reviewer: Prof. Dr. Markus Nüttgens

Second reviewer: Niklas-Kim Laerum

Hamburg, October 3, 2025

Abstract

Process mining is a data-driven approach to analyze and improve business processes based on event logs. A central artifact of process mining is the process model, a directed graph that represents activities as nodes and edges represent directly-follows relationships between activities. These models are frequently explored by adding process variants step by step. For analysts to effectively compare evolving models, layouts must remain visually stable across versions, preserving the mental map of users while clearly showing differences.

Despite the importance of this requirement, no dedicated dynamic layout algorithm for process mining process models has been available. This thesis introduces *Dynamic Dagre*, an enhanced version of the established *Dagre* algorithm, designed to support stable dynamic layouts. The modifications focus on minimizing disruptive changes by preserving relative positions of nodes, while remaining fully integrated into *Dagre*'s original architecture. A novel concept of a *core path* is introduced: a designated process variant, such as the most common or target process, that remains centrally fixed and exceptionally stable, serving as an anchor for surrounding variants.

The development was structured using a Design Science Research procedure model, with iterative cycles of problem awareness, suggestion, development, and evaluation guiding the evolution of the algorithm. This ensured both theoretical grounding and practical relevance.

The algorithm was evaluated against the baseline *Dagre* implementation and the commercial tool *Celonis* using established quality and stability metrics. Results demonstrate substantial improvements in stability, *Dynamic Dagre* consistently maintained layouts across graph versions, with the core path often preserved pixel-perfectly. Visual quality also improved in terms of edge crossings, while tradeoffs emerged for measures such as uniform edge lengths. Overall, the algorithm significantly advances dynamic layouting for process mining process models, providing an openly available solution that benefits exploration and conformance checking tasks.

Zusammenfassung

Process Mining ist ein datengetriebener Ansatz zur Analyse und Verbesserung von Geschäftsprozessen auf Basis von Ereignisprotokollen. Ein zentrales Artefakt des Process Minings ist das Prozessmodell, ein gerichteter Graph, in dem Aktivitäten als Knoten dargestellt werden und Kanten das Aufeinanderfolgen von Aktivitäten repräsentieren. Diese Modelle werden häufig explorativ untersucht und schrittweise durch das Hinzufügen von Prozessvarianten erweitert. Damit Analysten die sich entwickelnden Modelle effektiv vergleichen können, müssen Layouts über verschiedene Versionen hinweg visuell stabil bleiben, das mentale Modell der Nutzer bewahren und gleichzeitig Veränderungen klar hervorheben.

Trotz der Relevanz dieser Anforderung existierte bislang kein dedizierter dynamischer Layout-Algorithmus für Prozessmodelle. Diese Arbeit stellt *Dynamic Dagre* vor, eine erweiterte Version des etablierten *Dagre* Algorithmus, die speziell entwickelt wurde, um stabile dynamische Layouts zu ermöglichen. Die Modifikationen zielen darauf ab, störende Veränderungen zu minimieren, indem relative Positionen der Knoten erhalten bleiben, während die Integration in die ursprüngliche Architektur von *Dagre* gewährleistet wird. Darüber hinaus wird ein neues Konzept eingeführt: der *Core Path*, eine ausgewählte Prozessvariante, etwa der häufigste oder der Zielprozess, der zentral fixiert und besonders stabil gehalten wird und so als visueller Anker für umliegende Varianten dient.

Die Entwicklung folgte einem Design Science Research Vorgehensmodell, bei dem iterative Zyklen aus Problembewusstsein, Vorschlag, Entwicklung und Evaluation die Entwicklung des Algorithmus steuerten. Dies stellte sowohl die theoretische Fundierung als auch die praktische Relevanz sicher.

Der Algorithmus wurde anhand von dem original *Dagre* sowie des kommerziellen Tools *Celonis* mit etablierten Qualitäts- und Stabilitätsmetriken evaluiert. Die Ergebnisse zeigen deutliche Verbesserungen in der Stabilität: *Dynamic Dagre* konnte Layouts über mehrere Graph-Versionen hinweg konsistent erhalten, wobei der *Core Path* oft sogar pixelgenau stabil blieb. Die visuelle Qualität verbesserte sich insbesondere hinsichtlich der Kantenkreuzungen, während bei anderen Kriterien wie gleichmäßigen Kantenlängen gewisse Zielkonflikte auftraten. Insgesamt stellt der Algorithmus einen bedeutenden Fortschritt für dynamisches Layouting von Prozessmodellen dar und bietet eine frei verfügbare Lösung, die insbesondere Erkundung und Konformitätsprüfung unterstützt.

Contents

1. Introduction	1
1.1. Motivation	1
1.2. Structure of this thesis	2
2. Theoretical Foundations	3
2.1. Process Models	3
2.2. Graph Layout	4
3. Procedure Model	7
4. Dagre	10
4.1. Reasons for Dagre	10
4.2. Structure of Dagre	10
5. Modifications to Dagre	14
5.1. Rank	14
5.2. Order	18
5.3. Core Path	27
6. Evaluation	35
6.1. Methodology	36
6.2. Visual Stability	38
6.3. Layout Quality	46
7. Discussion	52
7.1. Research Questions	52
7.2. Contributions	55
7.3. Limitations	56
8. Conclusion and Future Work	57
Appendices	59
Bibliography	63

List of Figures

2.1. Illustration of hierarchical layout: nodes are arranged into ranks (rows) and ordered horizontally within each rank.	5
3.1. A Design Science Research Approach by Vaishnavi and Kuechler.	8
5.1. Dagre's implementation to remove cycles	15
5.2. Layouts resulting from three different implementations of the <i>acyclic</i> call with identical input.	16
5.3. DFS that keeps relative order	17
5.4. Global orders resulting from <i>sweeps</i> with different directions and bias. Initial order shown for reference.	20
5.5. Antiparallel edges with and without grouping.	22
5.6. Function to compare the old order of two dummy nodes	23
5.7. Layout comparison when ignoring or maintaining relative order.	25
5.8. DFS to build initial order	26
5.9. Function to sort ranks within <i>sweeps</i>	27
5.10. Alignment for crossing non-inner segments with different biases.	28
5.11. Comparison of the same graph with and without an aligned core path	29
5.12. Simplified example of the <i>align</i> and <i>root</i> object.	31
5.13. Dagre's implementation of the vertical alignment step.	32
5.14. Modified implementation of the vertical alignment step with core path. . . .	34
6.1. Visual analysis of a Celonis layout.	36
6.2. Dagres and Celonis layout for the most common variant.	37
6.3. Second layout for each algorithm.	42
6.4. Third layout for each algorithm.	43
6.5. Fourth layout for each algorithm.	44
6.6. Fifth layout for each algorithm.	45
1. Function to make sure an edge is save to reverse	60
2. Full function to compare the old order of two dummy nodes	61

List of Tables

6.1. Graph drawing offset for the three algorithms on six versions of a dynamic graph.	39
6.2. Z-scores for the amount of edge crossings (cross#). Smaller values are better and highlighted in green. Larger values are red.	47
6.3. Z-scores for the minimum angle of edge crossings (crossRes). Larger values are better and highlighted in green. Lower values are red.	47
6.4. Z-scores for the minimum angle of edges incident to the same node (angularRes). Larger values are better and highlighted in green. Lower values are red. . . .	48
6.5. Z-scores for the standard deviation of edge lengths (uniEdge). Smaller values are better and highlighted in green. Larger values are red.	50
6.6. Overall quality score O for all algorithms with and without angular resolution. Larger values are better and highlighted in green. Lower values are red. . . .	51

1. Introduction

Process Mining is a data-driven tool for analyzing and improving processes. Most business activities today leave digital traces, regardless of whether they are executed by a human or a machine [Van11]. These records, called event logs, provide an objective and comprehensive representation of the actual process execution. In contrast, handmade models or subjective opinions might not accurately reflect reality, and thorough manual analysis is often not feasible for the scale and complexity of modern processes.

Process mining encompasses three fundamental approaches. Process discovery automatically creates process models from an event log, showing how processes actually flow. Conformance checking compares what actually happens against what should happen according to the target process. Process enhancement adds valuable information to existing process models, such as where bottlenecks occur or how resources are used.

1.1. Motivation

This thesis focuses on the automated process model generation aspect of process mining.

Process models derived from event logs typically contain numerous process variants, representing the different walks that cases can follow through a process. When all variants are displayed simultaneously, the resulting visualization becomes overwhelming and difficult to comprehend. Users struggle to extract meaningful insights from complex process models that show every possible execution walk at once [MRA10].

To address this challenge, analysts need the ability to explore process complexity incrementally. An effective approach is to begin with the most frequent process variants and progressively add less common variants step by step. This gradual exploration enables better understanding and comprehension by allowing users to build their mental model of the process systematically. However, as variants are added or removed during this exploratory process, it is crucial that the overall layout of the process model remains stable. Preserving the spatial arrangement of process elements maintains the user's mental map and prevents disorientation that would occur if the layout changed dramatically with each modification [Mis+95].

Currently, there appears to be no dynamic graph layout library or tool specifically designed for process models that can handle this incremental variant exploration while maintaining layout stability.

1. Introduction

The research questions for this thesis are as follows:

- RQ1 How can properties of process mining process models be leveraged to develop a dynamic layout creation algorithm?
- RQ2 Can established layout creation algorithms be modified to visualize dynamic process models?
- RQ3 How does the resulting layout quality and stability compare to other algorithms for visualizing process models?

To answer these questions I built *Dynamic Dagre*, a dynamic layout creation algorithm. During the development, three key directions of improvement emerged:

- (1) enhancing the general readability and aesthetic quality of layouts,
- (2) ensuring stability of layouts across different process model versions to support cognitive continuity, and
- (3) introducing the concept of a core path, a path which is emphasized in the layout by being centrally positioned and kept aligned.

This combination of improvements aims to provide not only visually appealing but also cognitively supportive layouts that facilitate process model analysis and interpretation.

1.2. Structure of this thesis

This thesis is structured as follows: Chapter 2 establishes the foundations of process models and graph layouts.

Chapter 3 introduces the procedure model guiding this research, describing how the design steps were structured and how they shaped the development of the proposed solution.

Chapter 4 then provides a comprehensive analysis of Dagre, the layout algorithm I selected as the foundation, detailing its architecture and identifying the parts where improvements can be seamlessly integrated.

Building on this foundation, Chapter 5 presents the core contribution: novel enhancements that enable Dagre to maintain layout stability across different states of a process model.

Chapter 6 evaluates the enhanced algorithm through quantitative metrics, examining layout stability and visual quality.

The implications and limitations of these findings are explored in Chapter 7, which discusses the tradeoffs and the potential impact on graph visualization applications.

Chapter 8 summarizes the contributions and reflects on the research outcomes

2. Theoretical Foundations

2.1. Process Models

Process models provide a formal representation of business processes by describing activities and their relationships. In the context of process mining, such models are discovered from event logs and serve as visual and analytical tools to explore, analyze, and evaluate real process executions.

In these models, each **activity** is represented by a node. A **case** corresponds to one execution of the process from start to end, and a **variant** groups together all cases that contain the same activities in the same order. Variants can therefore be expressed as directed walks through the graph, starting at a unique source node ("Start") and ending at a unique sink node ("End"). A walk is a sequence of nodes and edges, similar to a path, but without the requirement that nodes or edges be distinct, allowing the representation of cycles [BW10]. Since loops may occur, process mining models are not necessarily acyclic. Each activity is represented by exactly one unique node, even if it appears in multiple variants.

Although these models are related to established notations such as Business Process Model and Notation (BPMN), they are usually much simpler in structure. Unlike BPMN, process mining models typically contain only activities and their directly-follows relationships. Elements such as logical gateways, timers, or events are not considered in this thesis.

The resulting graph structure exhibits the following characteristics:

- It is directed and simple: no multiple edges exist in the same direction between the same pair of nodes. Antiparallel edges, however, are possible.
- It often contains fewer than 100 nodes in practice. For example, the BPI Challenge dataset contains over 252,000 cases but still only 42 distinct activities.
- Edges tend to outnumber nodes substantially in larger models, as the same activities can be connected in many different ways.

Process models in process mining can be understood as sets of directed walks through a graph, all sharing the same start and end node. Not every possible walk corresponds to an actual process variant, but every variant can always be represented as a walk.

2.2. Graph Layout

Graph layouts provide the visual structure through which process models are represented. A layout determines how nodes and edges are placed in two-dimensional space, and thus has a direct influence on the readability and interpretability of the model. Especially in process mining, where analysts must detect detailed deviations and compare differences, a clear and stable layout is essential.

Several types of layouts exist, ranging from force-directed layouts, which place nodes based on simulated physical forces, to more structured approaches such as hierarchical layouts. In process mining, the directly-follows graph is the most common representation, and hierarchical layouts are typically used to visualize it.

In a hierarchical layout, nodes are assigned to ranks, which represent their distance from the source node (“Start”) along a directed path toward the sink node (“End”)[STT81]. By default, ranks are arranged from top to bottom, and nodes within the same rank are ordered horizontally from left to right. Compared to force-directed layouts, hierarchical layouts impose stricter structure, which reduces visual clutter and emphasizes the inherent directionality of processes.

The layout can also be represented as a matrix, where rows correspond to ranks and columns indicate the order of nodes within a rank (see Figure 2.1).

This figure illustrates how the dagre layout algorithm (hereafter referred to as Dagre) organizes nodes into ranks and orders [dag].

Ranks correspond to rows in the matrix, representing the vertical placement of nodes and edges from source to sink. Each node is assigned a rank, e.g., node C is placed at rank 4. Ranks only for edges appear between ranks with activity nodes to accommodate edge routing.

Orders define the horizontal arrangement of nodes and edges within the same rank. For example, in rank 4 of Figure 2.1, the order is: C, B, (A,End), where (A,End) denotes the edge from node A to node End.

Sometimes, order is described in index form, for instance, “node B has order 1” means B occupies the position indexed with 1 in the ordering of its rank. Similarly, “node C has rank 4” means node C is placed in the rank with index 4.

Finally, the concept of **relative rank** describes how nodes compare across layouts. For example, if the layout in Figure 2.1 were the baseline and a new layout placed node A at rank 6 while keeping node C at rank 4, their relative rank would have changed.

2. Theoretical Foundations

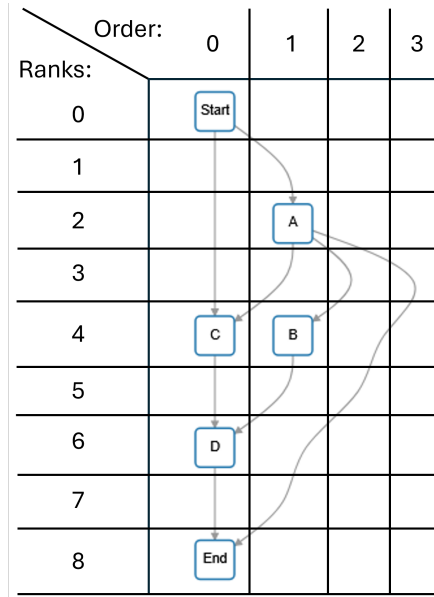


Figure 2.1.: Illustration of hierarchical layout: nodes are arranged into ranks (rows) and ordered horizontally within each rank.

A particular challenge arises when the graph changes over time. **Dynamic graph layouts** focus on maintaining stability across different versions of a graph [FT08]. In process mining, this situation occurs naturally: user often begin by inspecting only the most frequent variant of a process and then iteratively add further variants to the model. With each step, the graph grows in size and complexity, making it a prime candidate for dynamic layout techniques.

The central goal of dynamic layouts is to preserve the mental map of the user. That is, the visual structure of the graph should remain as consistent as possible between successive states, so that differences are easy to detect and orientation is maintained. Preserving mental maps often requires minimizing movements of nodes and edges. In the terminology of Misue et al. [Mis+95], this involves keeping orthogonal ordering stable, i.e., retaining the relative horizontal and vertical placement of nodes.

Dynamic layouts can be classified as **online** or **offline**:

In the **online** case, the algorithm only has knowledge of the current state and its predecessors. This is the scenario addressed in this thesis, where the goal is to preserve stability as the model evolves incrementally, starting from the initial variant and extending step by step.

In the **offline** case, all possible graph states are known in advance. While this makes it possible to compute globally optimized layouts across all versions, it is not applicable in the context of process mining. Analysts rarely work with the complete set of variants. Instead, they typically focus on the most common behaviors or apply filters to examine a limited selection of variants.

2. Theoretical Foundations

Research on the visualization of dynamic graphs exists, but it is limited in scope and appears to be entirely unrelated to process mining. Most approaches focus on force-directed layouts [FT08; Fen+12; GBG12; Hay+13] or on general undirected graphs [BM12]. These methods are not well-suited for process models, which are directed and have a natural flow that makes hierarchical layouts the more appropriate choice.

In practice, tool support for dynamic graph layout is sparse. Widely available layout libraries rarely include functionality for handling dynamic changes. The few solutions advertising such features were either outdated and no longer usable [dyn], or restricted behind commercial licensing models [yWo].

To the best of my knowledge, no existing algorithm or tool directly addresses the problem of stable dynamic layouts for process mining process models. This gap motivated the development of such an algorithm.

3. Procedure Model

In Design Science Research(DSR), procedure models provide a structured way to plan, document, and reflect on the development of artifacts. They ensure that the research process is both systematic and transparent, linking practical problem-solving with scientific rigor.

For this thesis, I follow the Design Science Research procedure model proposed by Vaishnavi and Kuechler [VK04]. This model is well established in information systems research and particularly suitable here, as it reflects the iterative nature of my approach: identifying problems in existing graph layout algorithms, developing and refining solutions, and evaluating them against practical and scientific criteria.

The model is illustrated in Figure 3.1. In the following, I map each of its phases to the specific steps of my work on developing and evaluating Dynamic Dagre.

The overarching goal of this thesis was to improve layout generation for process mining process models by stabilizing graph layouts when adding and removing case variants to the graph. To achieve this, I modified the graph layout algorithm Dagre, transforming it into a dynamic version that preserves existing structures wherever possible.

In the **Problem Awareness** step, I identified shortcomings of the original algorithm. The first issue was that the Dagre algorithm frequently reassigned rankings to nodes, even when maintaining the previous ranking would have been possible. This unnecessary re-ranking caused nodes to move significantly between layouts, disrupting visual stability. Other problems became apparent during later iterations, such as edge crossings or a lack of consistency in node placement when new elements were added. Each of these issues defined a concrete aspect for improvement.

In the **Suggestion** step, possible modifications to the algorithm were conceived for each identified problem. For instance, when the issue of unnecessary re-ranking was identified, I initially considered altering the ranking procedure by using the previous ranking for the current layout if the nodes remained the same. During development, however, I discovered that directly overwriting the ranking caused errors in later stages of the algorithm, that were difficult to trace. This experience led me to reevaluate the problem and realize that the ranking itself was not the root cause. Rather, the step to break cycles preceding the ranking had a decisive influence on how ranks were determined.

From this point on, a guiding principle emerged: to intervene as little as possible in the original Dagre implementation. Instead of imposing drastic changes, the focus shifted to targeted, minimal adjustments that preserved the algorithm's overall behavior while improving stability. In this way, suggestions were not abstract design ideas but concrete, code-level hypotheses about which specific parts of the algorithm could be altered to achieve the intended effect.

3. Procedure Model

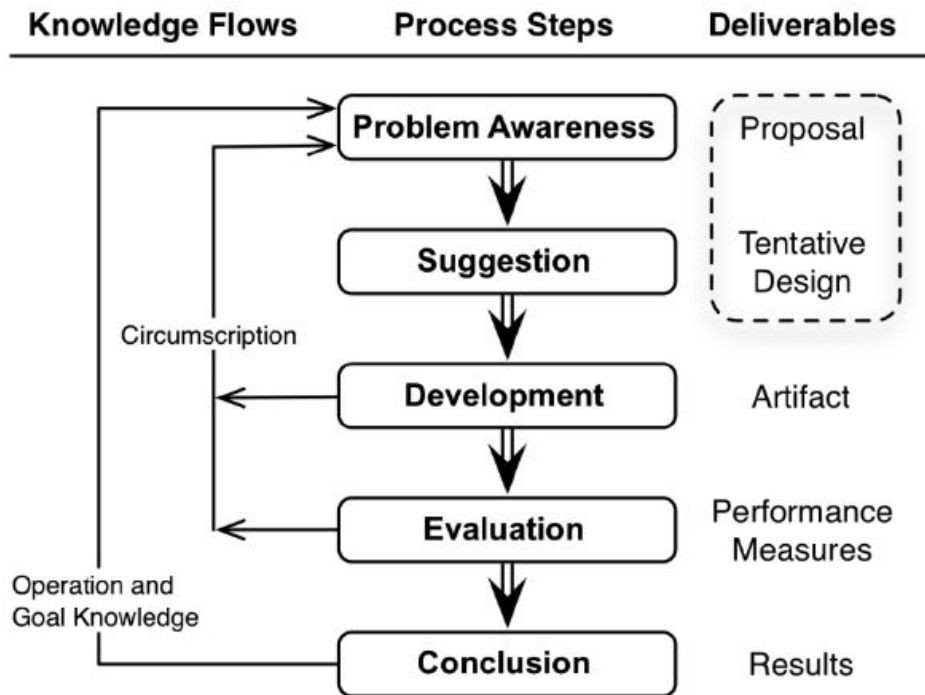


Figure 3.1.: A Design Science Research Approach by Vaishnavi and Kuechler.

The **Development** step consisted of implementing the proposed modifications directly in the Dage code base. Following the principle of minimal intervention, changes were deliberately kept as non-invasive as possible. In practice, this often meant stabilizing parts of the algorithm where randomness or implicit choices affected the outcome, such as tie-breaking rules or traversal order.

Continuing with the ranking stability example: the algorithm uses a depth-first search (DFS) to identify and reverse edges that create cycles. While there were explicit criteria for edge selection, the traversal order of the DFS strongly influenced which edges were chosen. I implemented logic that guided this selection to preserve relative rankings while still satisfying the original constraints, thereby avoiding errors in later steps.

A simpler adjustment also contributed to ranking stability. Originally, the DFS was started from all nodes in an arbitrary order determined by the underlying data structure. I modified this so that the source node was always chosen first, ensuring that it consistently occupied the top position in the ranking.

The artifacts created in this step are the modifications to the code and together combine into the novel *Dynamic Dage* algorithm.

3. Procedure Model

The **Evaluation** step combined immediate qualitative testing during development with a structured quantitative assessment of the final algorithm.

In the early stages, evaluation was carried out informally after each modification. Layouts were generated for representative process mining graphs, and I visually inspected them for whether the layout changes aligned with the intended improvements. This rapid feedback loop was essential for steering further development, as new problems often became visible only once the modification was put into practice.

Once the development was complete, the final version of the algorithm was evaluated quantitatively. Here, established metrics from the graph drawing literature were applied to assess two key aspects: the visual stability of layouts across changes and the intrinsic quality of individual layouts. For context, the results were also compared against other widely used layout algorithms. This ensured that the evaluation did not rely on subjective impressions alone, but was grounded in measurable criteria and relative performance.

The **Conclusion** step summarized the findings from development and evaluation. The final algorithm, *Dynamic Dagre*, proved to be significantly more stable and comparable in terms of layout quality to established algorithms. It demonstrated that carefully chosen modifications to an existing algorithm can achieve meaningful improvements for dynamic graph drawing in process mining. At the same time, the evaluation helped to identify potential applications for the algorithm as well as its limitations, providing a perspective on its overall contribution. These insights form the basis for future refinements and open avenues for further research.

4. Dagre

Dagre is an open source JavaScript library that implements a hierarchical layout algorithm for directed graphs based mainly on the approach described by Gansner et al. in "A Technique for Drawing Directed Graphs" [Gan+93]. This chapter presents Dagre's architecture and algorithmic approach, building the basis for the dynamic layout enhancements.

4.1. Reasons for Dagre

Dagre was chosen as the basis for a number of reasons:

First, Dagre is already in use as the layout creation algorithm for JointJS, the visualization framework currently utilized by Process.Science for rendering interactive visualizations of process models [joi]. This existing integration allowed for convenient testing and evaluation of the enhancements in a real-world system.

Second, Dagre implements a sophisticated hierarchical layout approach for directed graphs, fitting for process models where there is a general direction and hierarchy. The algorithm is structured as a sequence of modular steps, each of which can be modified independently, enabling incremental and isolated enhancements.

Third, as an open source project licensed under MIT, Dagre is freely accessible and permits redistribution, making it suitable for integration and further distribution with the enhancements for dynamic layouts. This enables the incorporation of enhancements within the existing framework, eliminating the need to develop a new algorithm from scratch.

4.2. Structure of Dagre

Dagre follows a three-phase pipeline to generate hierarchical layouts. First, it assigns ranks (vertical levels) to nodes. Second, it orders nodes and edges within each rank to minimize edge crossings. Third, it calculates the coordinates for all elements of the graph. Throughout this process, the algorithm temporarily modifies the graph to fit the requirements of the following calculations.

The data structure for graphs used by Dagre only allows for directed graphs. Although it accepts cycles in the input, its layout algorithm requires directed acyclic graphs (DAGs). The algorithm temporarily removes self-loops and breaks cycles by reversing edges in a feedback arc set (FAS) [ELS93].

4. Dagre

These modifications persist throughout the ranking and ordering phases, then are reversed to restore the original graph structure. These edge reversals significantly impact the final layout, as different feedback arc sets lead to different hierarchical structures. I developed two enhancements to the FAS selection algorithm: one to improve visual stability in dynamic graphs and another to improve overall layout quality. Both modifications are detailed in Section 5.1.

Following the transformation to a DAG, all nodes are assigned ranks that will determine the vertical positions in the layout. The rank of a node always needs to be higher (towards the bottom of the layout) than the rank of all of its predecessors. Initially, a simple but valid ranking is calculated with the longest path algorithm [Kha11] and then improved with the ranking algorithm from "A Technique for Drawing Directed Graphs" [Gan+93], by decreasing ranks and thus shortening edges.

With a DAG as input all ranks are well defined: every node is assigned the lowest (topmost) possible rank ≥ 0 while still being placed higher than all of its predecessors. Ambiguity only arises when the input graph contains cycles. In that case, the choice of reversed edges in the feedback arc set influences the resulting ranking. For this reason, the ranking algorithm itself remains unchanged in this work, while improvements are introduced in the construction of the FAS. These refinements are sufficient to ensure stable ranks across layouts.

Then, the horizontal order of the nodes and edges is calculated for each layer, respectively. For this phase, all edges are represented by chains of dummy nodes with one of them per rank that the edge spans. For example, if there is an edge between a node on rank 2 and a node on rank 6, it will be represented by three dummy nodes on rank 3, 4 and 5, respectively. Furthermore, between each adjacent pair of ranks with nodes, there is a rank with only dummy nodes, causing even the shortest edges between two different nodes to be represented by at least one dummy node.

Dagre creates four global orders, the combined orders across all ranks, and compares them by their amount of edge crossings.

It is not feasible to find the optimal global orders due to the many possible ones and the time it takes to count their edge crossings. For a rank with n nodes, including dummy nodes, there are $n!$ possible orders. With r ranks there are roughly $O(n!^r)$ possible global orders.

Instead, a simple initial order is computed using a depth-first search (DFS). For each rank, the algorithm maintains a counter of discovered nodes. When a node is first encountered, it is assigned the current counter value of its rank, after which the counter is incremented.

Then, a heuristic based on barycenters [JM97] uses this initial order to efficiently approximate a global order that minimizes edge crossings. The graph is traversed in four "sweeps" alternating between a forward direction, lowest rank to highest rank, and a backwards direction as well as alternating between a right bias and a left bias.

4. Dagre

In each sweep, the algorithm iterates over all ranks, sorting the nodes within each rank according to their barycenters. The barycenter of a node is defined as the weighted average of the positions of its immediate predecessors in their respective order. For a node n_s on rank r_s , its predecessors are defined as all adjacent nodes on rank $r_t = r_s - 1$ in the forward direction, or on rank $r_t = r_s + 1$ in the backward direction, regardless of the edges directions. The positions of the predecessors are weighted by the corresponding edge weights. In all graphs considered in this thesis, a uniform edge weight of 1 is used. Within each rank nodes with identical barycenters are ordered according to the initial order.

This process produces four global orders. However, two of those are merely mirrored versions of the others and never utilized. These mirrored versions arise from the left or right bias which determines for nodes with identical barycenters if the node, with the lower initial order, gets a lower (left bias) or higher (right bias) order in that sweep. If two nodes within the same rank have different barycenters the order of those was determined by that bias at the point where their two paths split from one another. This happened at the source (forward direction) or the sink (backwards direction) the latest. These mirrored versions are never utilized because only a global order with less edge crossings is chosen over the previous one.

In contrast, the forward and backwards directions yield different global orders on non-trivial graphs, with varying numbers of edge crossings and distinct layout. Edge crossings tend to occur later in the sweeps, so deeper in the graph (higher ranks) for the forward direction and closer to the source (lower ranks) for the backward direction.

In my modified version, I refined the use of the bias so that it produces four genuinely distinct global orders, resulting in a reduced or at worst equal amount of edge crossings. Furthermore, the layout of the previous graph is taken into account when constructing the initial order and when the heuristic sorts the ranks, increasing visual stability. A detailed description of the modifications can be found in Section 5.2.

The final major step assigns coordinates to nodes, including dummy nodes and thus edges, based on their rank and order.

The vertical positions are derived directly from the ranks. The "rankSep" is a parameter setting the padding between ranks. Each rank has a height that is calculated as the maximum height of all nodes in that rank. Every node is vertically centered within its rank.

Horizontal positioning, by contrast, is more complex. Dagre implements the algorithm proposed in "Fast and Simple Horizontal Coordinate Assignment" [BK02], which applies a heuristic approach.

Nodes are aligned to either their median upper neighbor (upwards alignment) or median lower neighbor (downward alignment) with respect to the order. If the median neighbor is ambiguous due to an even amount of neighbors, a bias to either the left or to the right decides. The four combinations of upwards or downwards alignment and left or right bias leads to four distinct vertical alignments.

4. *Dagre*

Each of these alignments is then converted into a block graph, in which a block represents a chain of aligned nodes. These graphs are then compacted by assigning the blocks, and thus all nodes it consists of, horizontal positions as far left or right as possible according to the bias.

Finally, the four block graphs are merged to balance the directional and bias-specific tendencies. Each node is assigned the average of its two median horizontal positions, yielding the final coordinate assignment.

To better support process mining visualizations, Section 5.3 presents modifications in which a path representing a core process variant can be specified. All nodes on this path are consistently aligned to highlight the main flow.

5. Modifications to Dagre

Dagre provides a solid foundation for hierarchical layouts, but process models pose additional requirements that go beyond its standard configuration. In the course of this work, several targeted modifications were developed to better support these requirements. These focus on refining the ranking and ordering phases of the algorithm, as well as introducing the concept of a *core path* that remains visually central and stable throughout different layouts. The following sections describe these modifications in detail.

5.1. Rank

One of the very first steps of the Dagre algorithm is to transform the input graph into a directed acyclic graph (DAG) by removing any existing cycles. Dagre's approach to this is based on Gasner et al. who propose a depth-first search to construct a feedback arc set, a collection of edges whose reversal will transform the graph to a DAG [Gan+93]. The *FAS problem* asks for a *minimum* set of edges and it appears reasonable to reverse as few edges as possible, but a minimum FAS is NP-hard to construct. Furthermore, Gasner et al. compared their simple DFS-based approach to a heuristic that preferentially reverses edges participating in many cycles and found the DFS method to yield more stable layouts.

The code is shown in Figure 5.1 and described here. Three objects keep track of the current state: "fas" is an array holding all edges that will be reversed, "stack" contains all nodes in the current branch of the DFS, "visited" contains all nodes visited by the DFS, it persists throughout the whole acyclic call for one layout. The stack is used to detect cycles. Line 17 continues the DFS for all successors of the current node, except for successors already on the stack (line 18). A successor being on the stack means the current DFS branch traversed through that node already and is about to do so again, indicating a cycle. The edge between such a successor and the current node is then added to the "fas" array (line 19).

This DFS is started from all nodes, guaranteeing every node is reached, even disconnected subgraphs, although those are not possible in typical process models. The "visited" object assures "dfs" is executed only once per node.

5. Modifications to Dagre

```
1 function acyclic(g) {
2   let fas = dfsFAS(g);
3   fas.forEach(e => {
4     e.reverse();
5   })
6 }
7
8 function dfsFas {
9   let fas = [];
10  let stack = {};
11  let visited = {};
12
13  function dfs(v) {
14    if (visited[v]) return;
15    visited[v] = true;
16    stack[v] = true;
17    outEdges(v).forEach(edge => {
18      if (stack[edge.target]) {
19        fas.push(edge);
20      } else {
21        dfs(edge.target);
22      }
23    });
24    delete stack[v];
25  }
26
27  g.nodes().forEach(dfs);
28 }
```

Figure 5.1.: Dagre's implementation to remove cycles

Gasner et al. propose "[...] starting from some source or sink nodes if any exist.", to keep the "natural order" [Gan+93, p. 6]. Dagre instead starts the search from every node, thus potentially losing this natural order and possibly creating layouts where the source is not on the topmost rank.

I added "*g.sources().forEach(dfs);*" in line 26, having noticed the problem that arose from Dagre's implementation. I kept *g.nodes().forEach(dfs);* in, otherwise graphs without sources, where every node has at least one incoming edge, would cause errors and could not be laid out.

Figure 5.2 illustrates the resulting layouts of different acyclic graphs. The layout in (a), produced by the original Dagre implementation, the source node *Start* is placed on the second rank. This occurs because the DFS begins at node *D*, continues through *E*, and then explores the successors of *A*. Since one of *A*'s successors is *D*, which is still on the stack, the edge (*A*, *D*) is added to the feedback arc set and internally reversed.

The resulting layout in (a) is unintuitive, as the *Start* node does not appear at the top but is instead surrounded by other nodes in multiple directions, disrupting the expected top-to-bottom flow. Removing nodes *B* and *C* makes this problem even more apparent.

5. Modifications to Dagre

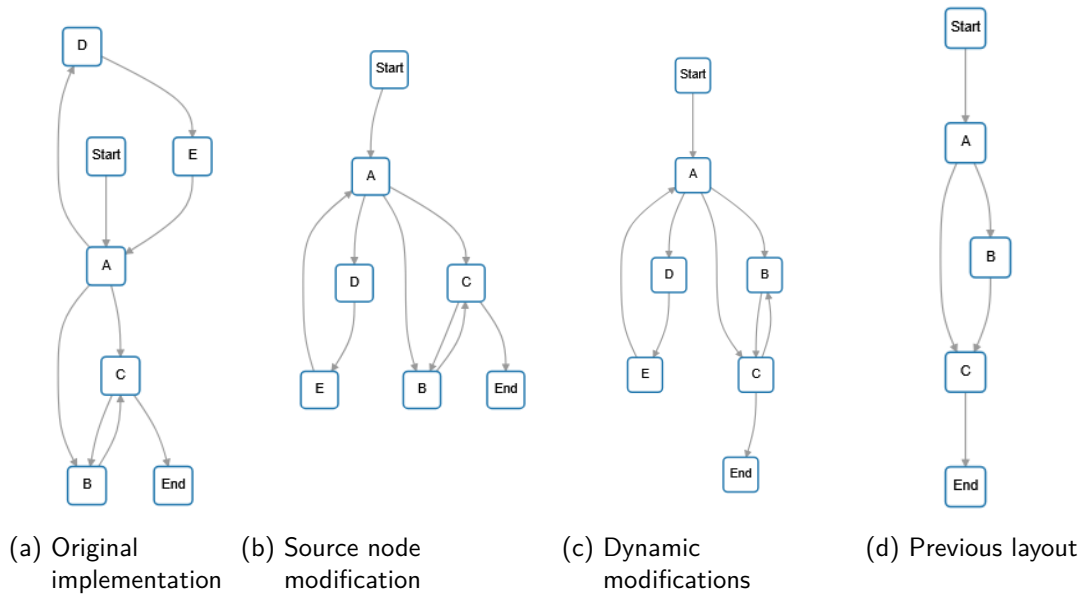


Figure 5.2.: Layouts resulting from three different implementations of the *acyclic* call with identical input.

The layout shown in Figure 5.2 (b) is the result of my first modification, starting the DFS from source nodes initially. The *Start* node is at the top, *A* is clearly visible as the first activity in all walks. Process models with exactly one source will always have *Start* as the lone topmost node with this implementation. The subgraph of $A \rightarrow D \rightarrow E \rightarrow A$ is positioned intuitively, following the general flow downwards until it loops back up to *A*.

This following modification is specifically for dynamic graphs, aiming to keep the relative ranks between nodes. Figure 5.2 (d) shows the previous state of the graph. Node *B* is on a lower rank than *C*, it is positioned above its successor. The relevant change from the graph in (d) to the one shown in (a)-(c) is the new edge (C, B) . *D*, *E* and their edges are not of interest for this modification.

In the implementations up until now the node discovered first by the DFS between *B* and *C* is assigned to a lower rank. If *C* is discovered first this would change their relative rank from the previous graph, disrupting the mental map. This happened in the layouts shown in (a) and (b).

I implemented a modified version of the DFS, shown in Figure 5.3, which takes the previous ranks of the nodes into account when constructing the FAS. The condition in line 6 remains unchanged from the original algorithm. In addition, a new condition is introduced: an edge (u, v) is added to the feedback arc set if it points backwards in the previous layout and the target node *v* is still reachable from a source node without this edge.

5. Modifications to Dagre

```
1  function dfsDynamic(v) {
2    if (visited[v]) return;
3    visited[v] = true;
4    stack[v] = true;
5    g.outEdges(v).forEach(edge => {
6      if (stack[edge.target]) ||
7        (v.oldRank > edge.target.oldRank) &&
8        isReachedFromStartWithoutEdge(edge) {
9        fas.push(e);
10     } else {
11       dfsDynamic(edge.target);
12     }
13   });
14   delete stack[v];
15 }
16
17 function isReachedFromStartWithoutEdge(edge) {
18   // checks if there is a path from a source node to edge.target
19   // without using edge itself
20 }
```

Figure 5.3.: DFS that keeps relative order

To evaluate this condition, a helper subroutine "isReachedFromStartWithoutEdge" is used. It performs a reverse depth-first search starting from the target node and traversing predecessors until a source node is reached. The implementation of this subroutine is shown in Figure 1 in the appendices.

The new condition extends the algorithm by marking edges for reversal even before an actual cycle is discovered. In many cases, a backwards edge is indeed part of a cycle, as nodes are required to have a higher rank than their predecessors, and a lower-ranked node is therefore likely to be an (indirect) predecessor. For example, preserving the relative order of nodes B and C demonstrates this behavior.

It should be noted that this approach may add edges to the FAS that are not part of a cycle. Nevertheless, in practice this leads to stable layouts across multiple model versions, which is highly desirable in process model visualization.

To avoid unwanted reversals, an edge (u, v) is not reversed if v is only reachable from a source through this very edge. In such cases, v must always lie later on a path from Start to End than u . This is equivalent to an activity v occurring strictly after activity u in every process variant currently examined, and the edge (u, v) must therefore be preserved in its original direction.

5.2. Order

The ordering phase determines the relative positions of nodes within the same rank. This step directly impacts the amount of edge crossings and significantly influences the layout quality and stability. Dagre aims to minimize edge crossings by applying heuristics as described in Section 4.2.

I introduced three modifications to the ordering phase. I identified a flaw in the original implementation that restricted the set of global orders discovered. With the first modification a larger set of valid orders becomes available for comparison, enabling the algorithm to select layouts with fewer crossings.

The second modification treats multiple edges between the same two nodes as one for the ordering phase. This reduces visual clutter and at the same time increases the range of order variations that can be considered, since the amount of elements per rank effectively decreases.

To enhance stability in dynamic layouts, the third modification introduces a mechanism that prioritizes the preservation of orderings from previous layouts. This helps maintain the user's mental map across different versions of the process model, reducing disorientation when models evolve.

Switching Bias

The four *sweeps* over the ranks of the graph described in Section 4.2 generate two distinct global orders and two mirrored versions of those. A sweep has to generate a global order with fewer edge crossings to be chosen over the previous one, thus the mirrored versions are never utilized. With my modification the bias is fixed to right in two versions, and changes between left and right within the other two *sweeps*, resulting in four distinct layouts.

Figure 5.4 visualizes an initial order and the four global orders resulting from Dagre's original *sweeps*, as well as a result from using this modification. Its important to keep in mind that edges are internally represented by chains of dummy nodes and that there are ranks in between the visual nodes. These graphs all have nine ranks, five of them with named nodes and partially dummy nodes as well, the other four ranks are for dummy nodes only. Every sweep iterates over all nine ranks and sorts them based on the previous rank and the initial order.

(a) shows the initial order that was constructed by a DFS, assigning each node its order in order of discovery per rank. The leftmost node per rank has order 0, the next node order 1 and so forth. In this example the DFS starts at the only source node *Start*, assigning it order 0. *Start* has two successors where the DFS can continue, here the edge $(Start, A)$ was chosen, assigning its dummy node $(Start, A)_0$ order 0. That dummy node has only *A* as successor, *A* gets order 0.

5. Modifications to Dagre

Now there are three successors to choose from, the DFS went towards C : the dummy node in between $(A, C)_0$ gets order 0 on its rank. There are no more branching options until end is reached, then the DFS continues from A towards B . (A, End) follows and lastly $(Start, C)$ is traversed with all its dummy nodes $(Start, C)_0$, $(Start, C)_1$ and $(Start, C)_2$ being assigned the largest order in their ranks.

(b) is the result of the sweep in the forwards direction with a left bias. The sweep starts at rank 0, its only node being $Start$. Since there is only one node there is no sorting. The next rank contains the two dummy nodes $(Start, A)_0$ and $(Start, C)_0$. First their barycenters are compared, the average order of their immediate predecessors respectively: both have a barycenter of 0, since both their predecessor is $Start$ with order 0. As a tiebreak the initial order is used according to the bias: $(Start, A)_0$ has the lower initial order and is placed left due to the left bias.

Rank 2 includes the nodes A and $(Start, C)_1$ with distinct barycenters. A 's predecessor is $(Start, A)_0$ which was just assigned order 0, $(Start, C)_1$ follows from $(Start, C)_0$ with order 1. Due to A 's lower barycenter it is placed left with order 0.

Next is the largest rank with four nodes: three successors of A have a barycenter of 0, $(Start, C)_2$ is the only one with a different barycenter of 1, it will be placed last (rightmost). The other three nodes are sorted according to the initial order: $(A, C)_0$, $(A, B)_0$, $(A, End)_0$.

The barycenters, here written as $b(v) = x$, determine the whole order for rank 4: $b(B) = 1$, $b(C) = \frac{0+3}{2} = 1.5$, $b((A, End)_1) = 2$. Resulting in two edge crossings between this and the previous rank.

The last four ranks are straight forward and cause no more edge crossings.

For a human these edge crossings are trivial to fix, highlighting the suboptimal initial order and limitations of this heuristic approach.

(c) shows the result of the next sweep using the same direction but a right bias instead of a left bias. The ranks are iterated over the same as in the sweep detailed above with the same critical decisions leading to the same edge crossings. The only difference is that this layout is flipped along the vertical axis creating a mirrored version.

Changing the direction, in contrast, leads to a distinct layout as shown in (d) and (e). For the sweep producing these layouts the algorithm starts at the highest rank, containing only End . Backwards sweeps traverse through predecessors along incoming edges.

Nodes on the second to last rank are sorted by the initial order, next the rank containing D by the distinct barycenters. $(C, D)_0$ and $(B, D)_0$ are sorted according to the initial order while $(A, End)_2$ is placed right due to its barycenter. Here is the first difference to the layouts from forward sweeps, as B and C are swapped.

On the next rank $(A, C)_0$ is placed furthest to the left, since its initial order is lower than that of $(Start, C)_2$. But A is being placed further right than $(Start, C)_1$ due their barycenters, causing an edge crossings.

This global order, with one fewer edge crossings, would be chosen over the global order resulting from the forward sweep.

5. Modifications to Dagre

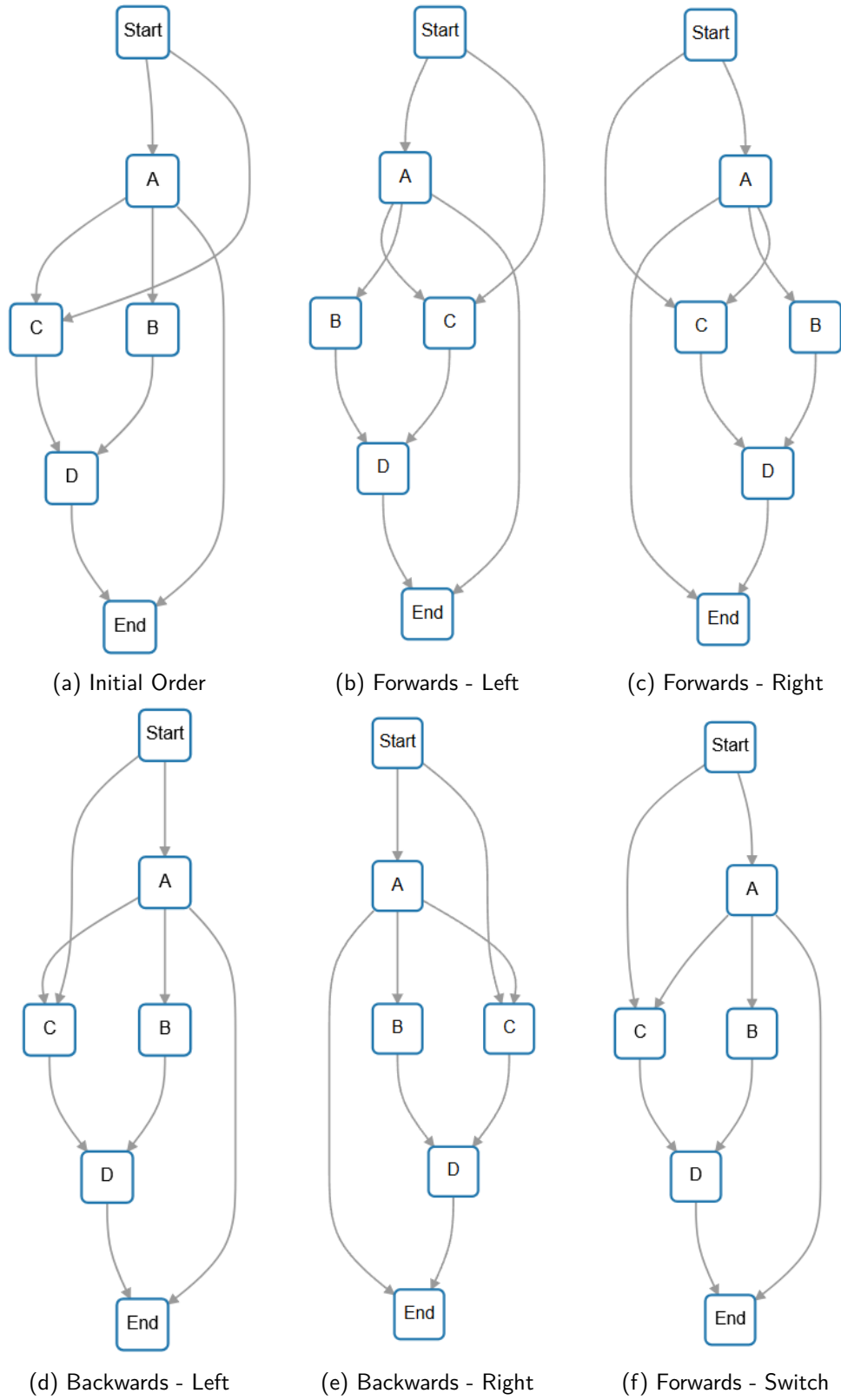


Figure 5.4.: Global orders resulting from *sweeps* with different directions and bias. Initial order shown for reference.

5. Modifications to Dagre

With my modifications to this algorithm the four *sweeps* create two layouts with a right bias and two layouts with a "switch" bias. This switch bias starts as a right bias and switches between left and right after every time a rank *used* this bias. *Using* meaning at least once in that rank two elements were compared via their initial order according to the bias, instead of a deciding barycenter. The results are four distinct layouts and thus possibly less edge crossings in the final layout.

(f) shows the result of the switch bias used in the forward direction. That sweep starts with a right bias, that is not *used* on the first rank with only *Start*. The second rank is sorted as in (c) with the edge $(Start, A)$ placed on the right. This *used* the bias changing it to a left bias for the next rank.

The rank following immediately after with A and $(Start, C)_1$ are sorted by their distinct barycenters, again as in (c), and the bias remains a left bias.

The switch bias shows its value on the fourth rank with the four edges: $(Start, C)_2$ is placed on the very left beneath its predecessor while the other three are sorted with a left bias by their initial order. Leading to the order of this rank as follows: $(Start, C)_2$, $(A, C)_0$, $(A, B)_0$, $(A, End)_0$.

The bias is switched to a right one again, but never used from here on, since the remaining ranks are trivially sorted by their barycenters. Resulting in zero edge crossings.

Grouping Antiparallel Edges

In a directed graph, two nodes may be connected by edges in both directions. That is, there exists an edge from node A to node B and another from B to A . These edges are referred to as antiparallel edges. For a graph layout it makes sense to draw such pairs of edges next to another in parallel. This highlights the loop they create and simplifies the layout, making it more understandable [Zho+13].

Parallel edges, two edges with identical source node and target node, never occurred in the models I worked with. Process models typically model only up to one edge, per direction, between two nodes. This edge represents all steps from activity A to activity B in the event log and aggregates associated metrics, such as cost and duration.

Antiparallel edges are rather common in process models derived from real-world event logs, multiple of them are present in the "BPI Challenge 2019" dataset as well as in other models I worked with. They can indicate repeated steps or reversible behavior for example. Dagre has no inherent support for antiparallel edges often causing these edges to be drawn individually with other edges or nodes in between.

This modification introduces support for antiparallel edges, ensuring they are always ordered as immediate neighbors.

5. Modifications to Dagre

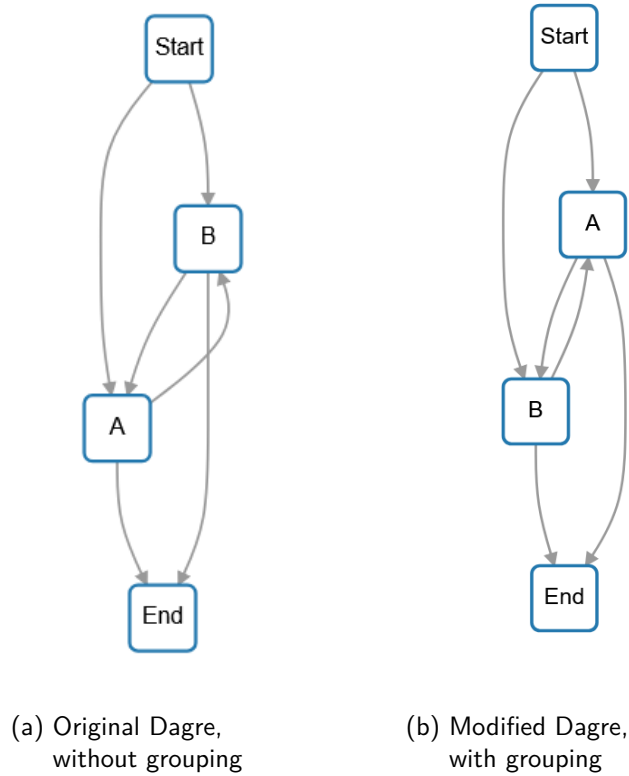


Figure 5.5.: Antiparallel edges with and without grouping.

The very basic graph in Figure 5.5 illustrates the problem with antiparallel edges using the original Dagre algorithm. The left layout in (a) introduces an unnecessary edge crossing while making the cycle between *A* and *B* less apparent. (b) shows a visually clean layout with the pair of antiparallel edges drawn next to one another, due to the grouping.

This modification is implemented within the *sweeps* mentioned above. Just before a rank is sorted, one edge per pair of antiparallel edges is temporarily removed, or rather its dummy node to be precise. After the altered rank is sorted, those removed edges are added back into the rank. They are placed directly next to their respective counterpart, receiving an order one higher than it. Thus visually highlighting the cycle and reducing clutter.

Additionally, having fewer elements to sort in a rank can lead to fewer edge crossings. For the example, in Figure 5.5 this modification causes the rank between *A* and *B* to effectively have three elements instead of four. Regardless of forwards or backwards direction, exactly one element has a distinct barycenter from the others. Without grouping there are $3! = 6$ possibilities to sort the remaining three elements, only two of which are explored. With grouping there are $2! = 2$ possibilities to sort the remaining two elements, both which are explored.

Maintaining Relative Order

The last modification regarding the order aims at keeping the relative order from the previous to the current layout. I implemented a function to compare two elements based on their order in the previous layout. This is used first to improve the initial order and second to sort ranks in the *sweeps*.

The examples shown and described in this chapter have highlighted the impact that the initial order has. A good initial order can significantly reduce edge crossings as well as ensure that the relative orders are kept the same. Since the algorithm to construct the initial order is a simple DFS and I did not want to completely overhaul this, the initial order will still have shortcomings. Thus the sorting of the ranks is also modified with the same goals. In combination these changes work well in practice.

```

1  function compareByOldOrder(oldNodes, a, b) {
2    if (!a or !b or !oldNodes) return 0;
3
4    let isForward = a.source === b.source;
5
6    if (isForward) {
7      targetRank = oldNodes[a.source].rank + 1
8    } else {
9      targetRank = oldNodes[a.target].rank - 1
10   }
11
12   let oldA = oldNodes.find(oldN => {
13     oldN.source == a.source and
14     oldN.target == a.target and
15     oldN.rank == targetRank
16   })
17   let oldB = oldNodes.find(oldN => {
18     oldN.source == b.source and
19     oldN.target == b.target and
20     oldN.rank == targetRank
21   })
22   if (!oldA or !oldB) return 0;
23
24   return oldA.order - oldB.order;
25 }
```

Figure 5.6.: Function to compare the old order of two dummy nodes

Figure 5.6 shows pseudocode of the aforementioned function *compareByOldOrder*, the real code is shown in the appendix in Figure 2. *compareByOldOrder* takes an object *oldNodes*, containing information such as rank and order of nodes in the previous layout, as parameter as well as two dummy nodes *a* and *b* to compare. Since this is a compare function that is passed to *.sort()* calls, it has to return a numerical value. Returning 0 means the the two elements are equal for the sake of this comparison. Returning a negative number means *a* should be placed before *b*, while returning a positive number places *b* before *a*.

5. Modifications to Dagre

compareByOldOrder is used where paths diverge to either incentivize the DFS for initial order to discover nodes according to the previous order or to sort ranks according to the previous order. Paths can only diverge from *proper* nodes, not dummy nodes, while the elements representing these diverging paths are dummy nodes. Visually speaking an edge never splits up into multiple edges, but a node can have multiple outgoing edges. Multiple ingoing edges also work for this if the current direction is backwards. These multiple edges are all represented by dummy nodes and can be sorted with this function.

First *compareByOldOrder* makes sure the parameters were all passed and contain the information needed, line 2 shows the simplified representation of that. In line 4 this algorithm determines the current direction. If both edges have the same source node it is the forward direction, otherwise backwards.

The source of an edge always has a lower rank than the target, if *a* and *b* have the same source these edges are outgoing edges from that same source and are thus positioned one rank below visually (source rank + 1 numerically). If this is not the case, the targets will be the same and these edges are incoming edges to the same target, positioned one rank above visually (target rank - 1 numerically).

The only exception to this are antiparallel edges, one of which was already internally reversed, where source and target are identical. Since the DFS for initial order is in the forwards direction this causes no problem in this case and grouping of antiparallel edges ensures that sorting the ranks works as intended as well. Looking at Figure 5.5 at the rank below *Start* or above *End* you can see how a common source (*Start*) indicates a forwards direction and identical targets (*End*) indicates a backwards direction.

To compare the orders of *a* and *b* relative to the previous layout, the corresponding nodes *oldA* and *oldB* have to be identified in *oldNodes*, the nodes of the previous layout. For *a* and *b* the algorithm iterates over all previous nodes comparing their sources and targets to *a*'s or *b*'s respectively (line 13-14, 18-19).

This alone might yield multiple matches since one edge can consist of more than one dummy node. To identify the correct one, the rank is taken into consideration. Given a forwards direction with *a* on rank r_a and its source node on rank $r_{a.source}$, we have $r_a = r_{a.source} + 1 = r_{b.source} + 1 = r_b$. Since the algorithm looks for the matching node in the previous layout the rank needs to be calculated relative to the source nodes rank in that layout.

Let $r_{p,v}$ denote the rank of node *v* in the previous layout. The rank where *oldA* and *oldB* will have to be is called *targetRank*, with $targetRank = r_{p,oldA} = r_{p,oldB} = r_{p,a.source} + 1$. The *targetRank* is calculated in lines 6-10 and this additional condition is checked in lines 15 and 20.

Since edges might have been added or removed from the previous layout to the current, it is possible that no corresponding elements exist at all and 0 is returned (line 22).

Lastly the order of *oldB* is subtracted from *oldA*'s and returned. This causes the relative order between *a* and *b* to be the same as between *oldA* and *oldB*.

5. Modifications to Dagre

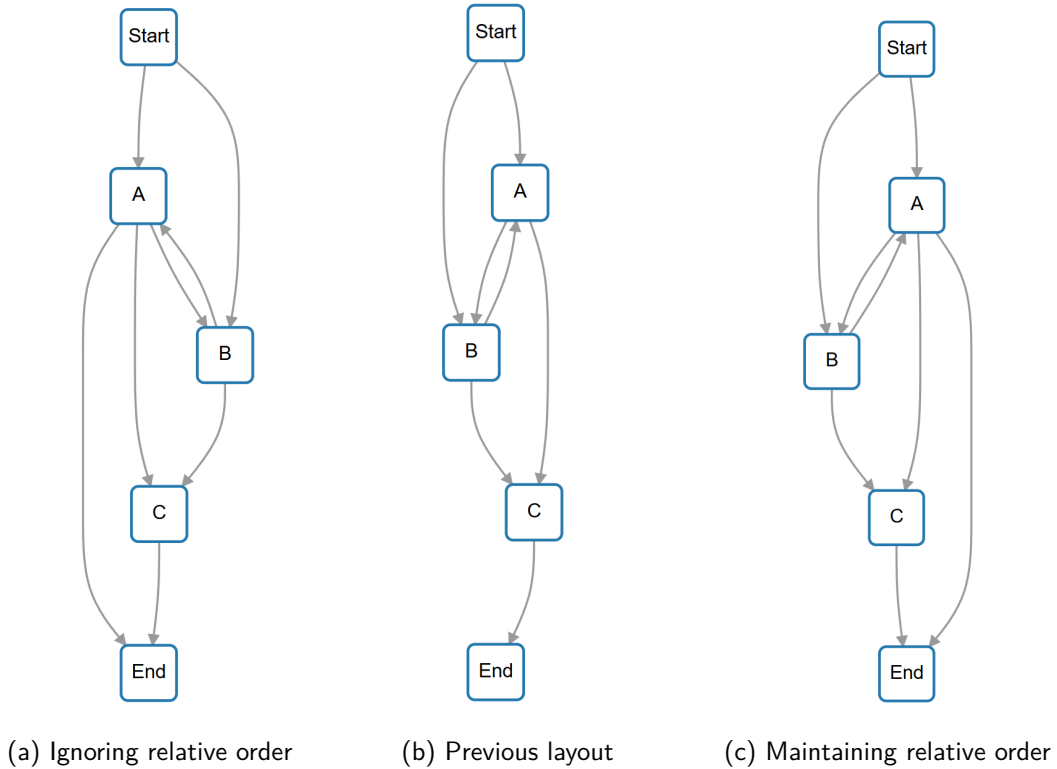


Figure 5.7.: Layout comparison when ignoring or maintaining relative order.

The DFS used to construct the initial order builds a matrix of all nodes, each rank is a row and the rows are built in the order in which nodes are discovered. Figure 5.8 shows the pseudocode for this. The first node discovered on a rank is pushed first, receiving order 0 (line 4). The next one pushed to that same rank will have order 1 and so forth.

Line 5-6 are the only modifications from the original algorithm. Before the DFS continues to the successors they are sorted with *compareByOldOrder* trying to maintain the previous order. Since any comparison to dummy nodes that were not present in the previous layout return 0, these new nodes can be placed anywhere in the initial order.

Looking at Figure 5.7 we can see the previous layout (b) in the middle, to the left (a) is the current layout without maintaining the relative order. (c) on the right uses information about the previous layout to maintain the relative order. Adding just one edge (A, End) leads to quite a bit of movement between layouts. Suddenly A is to the left of B instead of right, moving their edges along with them. The layout quality is still good without any edge crossings, but these changes make it harder to compare (b) to (a) especially when looking at them after another instead of next to one another.

(c) utilizes this sections modifications keeping the layout stable with all previous elements remaining at nearly the same positions.

5. Modifications to Dagre

```
1  function dfs(v) {
2      if (v.visited) return;
3      v.visited = true;
4      matrix[v.rank].push(v);
5      let successors = successors(v).toSorted(
6          (a, b) => compareOldOrder(a, b));
7      successors.forEach(dfs);
8  }
```

Figure 5.8.: DFS to build initial order

When the DFS begins at the source node *Start* the modifications make sure the edge towards *B* is its first successor which the DFS continues through, maintaining its order. Only when reaching *A* and choosing the next successor there is uncertainty as (A, End) is new, making its position in the sorted successors unpredictable. Let us assume the initial order for those successors is as follows: (A, B) , (A, C) , (A, End) . I deliberately left (B, A) out due to the grouping of antiparallel edges. (A, B) being left of (A, End) is guaranteed due to the modifications.

This initial order would always lead to edge crossings in the forwards direction. No matter if left or right bias, (A, C) would remain in the middle crossing either (A, B) or (A, End) . Here is where the second use of *compareByOldOrder* comes into play. Previously ranks were sorted by barycenter firstly and initial order secondly.

I introduced a metric in between: *compareByOldOrder*. This is shown in Figure 5.9. Lets look at the current example with this change in mind: The barycenters of (A, B) , (A, C) and (A, End) are still identical, but the new secondary metric ensures (A, B) will always be placed to the left of (A, C) . With a right bias (A, End) has to be to the right of (A, C) , since barycenter and relative order are indecisive for this comparison. Leading to an order of: (A, B) , (A, End) , (A, C) with no edge crossings.

5. Modifications to Dagre

```
1 function sortWithinRank(a, b, biasLeft) {
2     // return difference of barycenters, if it is not zero
3     let baryDiff = a.barycenter - b.barycenter;
4     if (baryDiff !== 0) {
5         return baryDiff;
6     }
7
8     // return difference of old ranks, if it is not zero
9     let byOldOrder = compareByOldOrder(a, b);
10    if (byOldOrder !== 0) {
11        return byOldOrder;
12    }
13    // return difference of initial order
14    // reversed if using right bias
15    return biasLeft ? a.initial - b.initial : b.initial - a.initial;
16 };
17 }
```

Figure 5.9.: Function to sort ranks within *sweeps*.

5.3. Core Path

A target process typically represents an idealized version of a process. Within a process model, this can be expressed as a path from *Start* to *End*, or as a small set of such paths. During the conformance checking step of process mining, event log data is compared against the target process. To support this task, it is beneficial to place the target process in the visual center of the model, with deviating process variants arranged around it.

My final modification of the Dagre layout algorithm therefore introduces the novel concept of a *core path*. A core path is a designated path in the graph whose nodes are vertically aligned, ensuring that it is prominently centered in the visualization. While this modification was developed with target processes in mind, it is not limited to that use case. For example, the most frequent path observed in the event log could also be designated as the core path.

The modification is integrated into the alignment phase of the Dagre algorithm. In the original algorithm, alignment serves to improve readability. Ideally, nodes that are directly connected should also be aligned, such that a shared horizontal position visually indicates a structural relationship. The algorithm constructs four alignment variants with either upward or downward alignment, each with left or right bias. For each variant, the algorithm iterates over all ranks and each node is checked against its neighbors (predecessors in upward alignment, successors in downward alignment). A neighbor is chosen based on the median position. In cases with two candidates, due to an even amount of neighbors, the algorithm attempts to align to the left or right one first, according to the bias, then the other.

5. Modifications to Dagre

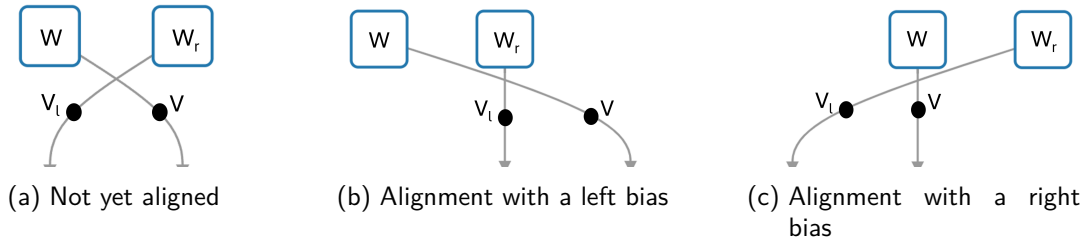


Figure 5.10.: Alignment for crossing non-inner segments with different biases.

A node v may only be aligned to a neighbor w if all of the following conditions hold:

1. v is not yet aligned to another node.
2. The edge (v, w) is not part of edge crossing with an *inner segment*.
3. No node from the current rank has been aligned to a node further to the right on w 's rank.

An *inner segment* is an edge connecting two dummy nodes. Any edge (x, y) crossing with an inner segment invalidates x and y from being aligned to one another. The source paper for this algorithm states that the layout should not have any edge crossings between two non-inner segments, "Since vertical inner segments appear to improve readability dramatically [...]" [BK02, p35]. The ordering with barycenters ensures there are no such edge crossings.

The third condition prevents crossed alignments. Lets say v is the current node, its neighbor w is evaluated as potential alignment. We assume no node has been aligned to w and (v, w) crosses no inner segment. Let there be a node v_l with $order(v_l) < order(v)$ aligned to w_r with $order(w_r) > order(w)$ (Figure 5.10 b). Due to their order (v, w) crosses (v_l, w_r) and the third condition prevents v from being aligned to w . In the alignment with the right bias v and w would be aligned instead of v_l and w_r . Because, in that case the rank of v and v_l is iterated over from right to left, aligning v to w and then barring v_l from aligning with w_r . This example is visualized in Figure 5.10.

As a result, chains or "blocks" of aligned nodes are formed. Each block is then assigned a horizontal position, and blocks are compacted as closely as possible while respecting minimum spacing constraints. This produces one compacted block graph per alignment variant. Finally, these variants are combined into a single layout: for each node its two median positions are taken from the four variants and averaged to get the final position. This causes every variant to have an impact on the final position while limiting the impact of "extreme" alignments that stray far from the others.

5. Modifications to Dagre

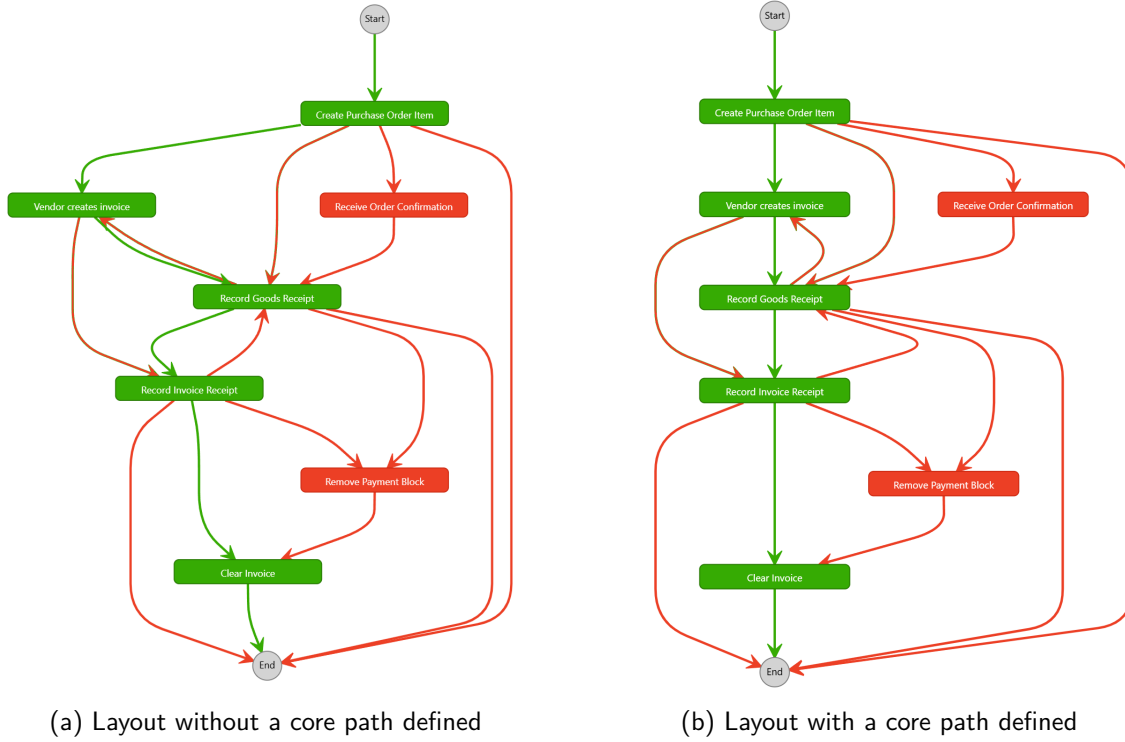


Figure 5.11.: Comparison of the same graph with and without an aligned core path

The core path modification ensures that the designated path is always aligned into a single block, whenever possible. Since a path is, by definition, acyclic [ZC06], each rank contains exactly one (dummy) node of the core path. This property guarantees that alignment can be applied consistently along the path.

In rare cases, the core path may be “severed” due to edge crossings with inner segments. When this occurs, the path is split into multiple aligned segments, which are then positioned consistently relative to one another. While such splits occur infrequently thanks to the stability of the dynamic layout, they may appear in large or highly complex process models.

When no splits occur, the core path forms a single block that is aligned in all four alignment variants. Consequently, it remains perfectly aligned in the final combined layout, ensuring that the chosen core path is always visually emphasized in the graph.

Figure 5.11 shows the effect of a core path. (a) uses all the modifications presented in this thesis, but there is no core path defined. (b) has the nodes and edges in green set as the core path, representing the target process. Both layouts have the exact same ranks and global order, but the results is still drastically different due to the alignment. The layout in (b) is way more structured, especially for the core path, highlighting the flow of the process model.

5. Modifications to Dagre

A benefit not visible in a single step is the stability between layouts. With the version in (b) the core path remains nearly static while the rest of the graph is added around it. (a)'s version is stable in the sense of rank and order but the positions still change, making it harder to understand changes from one graph iteration to the next. This visual stability is analyzed and highlighted next chapter, in Section 6.2.

The code for Dagre's alignment is presented in Figure 5.13.

The data structure of the parameter *layering* is represented as a matrix, where each row corresponds to a layer (rank) of nodes. The alignment determines the vertical order of the layers. With upward alignment, the layering begins at the topmost row, where the source node(s) are located, and proceeds downward through the matrix. In contrast, downward alignment starts from the lowest layer and traverses upward instead.

The bias determines the horizontal ordering of nodes within each layer. A left bias preserves the order of the nodes within a layer, whereas a right bias mirrors this order, so that nodes with the lowest order are positioned on the right-hand side. *conflicts* is an object holding all edge crossings with inner segments. Lastly *neighborFn* is a function that either returns the successors of a node in the upwards alignment or the predecessors in a downwards alignment.

Lines 2-12 initialize objects that hold certain values for each node. The *root* of a node is initially itself and is later set to the first node in the block it is aligned to. At the end there will be one block per distinct value in the *root* object. *align* keeps track of the neighbor a node is aligned to, initially every node is aligned to itself. *pos* is the index of a node within its rank and never changed in this algorithm, only queried. This can differ from its order if the current alignment uses a right bias.

A loop over all ranks starts on line 14 and continues to line 33.

For each layer this algorithm has a variable keeping track of the previous index a node was aligned to (*prevIdx*). The initial value of -1 ensures it is smaller than any real index. The value is updated whenever a node *v* is aligned to a node *w*, the new value is *pos(w)* (line 28). When determining if *v* can be aligned to *w* the previous index has to be smaller than the position of *w* to prevent crossed alignments (line 24).

Within the loop over all ranks is a loop iterating through all nodes on that rank (line 16), the current node is saved to the variable *v*. All of *v*'s neighbors are determined according to the *neighborFn* (line 17) and this iteration of the inner loop only continues if *v* does have at least one neighbor (line 18). These neighbors are sorted by position (line 21) and the mid point (*mp*) of the neighbors array is determined (line 22). This is either the index of the median neighbor or it is the average of the indices of the two median neighbors. For example, if there are 4 neighbors the *mp* will be 1.5, since 1 and 2 are the indices for the two median neighbors.

5. Modifications to Dagre

The *for* loop in line 23 is designed to check the conditions for alignment for both cases. If there is exactly one median neighbor, the floor of the midpoint $\text{Math.floor}(mp)$ will be equal to its ceiling $\text{Math.ceil}(mp)$ and only that neighbor is considered for alignment. If there are multiple median neighbors the floor and ceiling of mp will yield their indices and both are considered for alignment. The order in which these are tried is set by the left or right bias.

For v and its median neighbor w three conditions have to be met to be aligned: v can not be aligned already (line 25). No node on the current layer can already be aligned to a node with lower position than w , to prevent crossing alignments (line 26). The edge (v, w) can not have an edge crossing with an inner segment nor be an inner segment itself that crosses any other edge (line 27).

If no conditions are violated v is aligned to w by updating the *align* and *root* object. The syntax for the aligning can be confusing. Line 28: $\text{align}[w] = v$ means the node aligned to w is v . Next the *align* and *root* of v is set to the *root* of w . The root of all nodes who are aligned to another is the same, the root of the first node in this alignment. Furthermore, the value $\text{align}[v]$ for the last node v in the block is the very first node in that same block, forming a cycle.

Finally, after the all layers were iterated over the *root* and *align* objects are returned to calculate horizontal coordinates.

The following Figure 5.12 shows a simplified example of these objects. Dummy nodes would normally be present in these but are left out here for readability. In this example *Start*, *A* and *B* build an aligned block. *C* and *End* were not aligned to anything, perhaps due to edge crossings. In the next parts of the algorithm the three resulting blocks *Start*, *C* and *End* would be positioned and receive a horizontal coordinate each.

```
1      align: {
2          "Start": "A",
3          "A":     "B",
4          "B":     "Start",
5          "C":     "C",
6          "End":   "End"
7      }
8      root: {
9          "Start": "Start",
10         "A":     "Start",
11         "B":     "Start",
12         "C":     "C",
13         "End":   "End"
14     }
```

Figure 5.12.: Simplified example of the *align* and *root* object.

5. Modifications to Dagre

```
1 function verticalAlignment(layering, conflicts, neighborFn) {
2   let root = {},
3   let align = {},
4   let pos = {};
5
6   // The layering matrix is manipulated to
7   // generate different extreme alignments.
8   layering.forEach(layer => {
9     layer.forEach((v, order) => {
10      root[v] = v;
11      align[v] = v;
12      pos[v] = order;
13    });
14  });
15
16  layering.forEach(layer => {
17    let prevIdx = -1;
18    layer.forEach(v => {
19      let ws = neighborFn(v);
20      if (ws.length) {
21        ws = ws.sort((a, b) => pos[a] - pos[b]);
22        let mp = (ws.length - 1) / 2;
23        for (let i = Math.floor(mp), il = Math.ceil(mp); i <= il; ++i) {
24          let w = ws[i];
25          if (align[v] === v &&
26              prevIdx < pos[w] &&
27              !hasConflict(conflicts, v, w)) {
28            align[w] = v;
29            align[v] = root[v] = root[w];
30            prevIdx = pos[w];
31          }
32        }
33      }
34    });
35  });
36
37  return { root: root, align: align };
38 }
```

Figure 5.13.: Dagre's implementation of the vertical alignment step.

5. Modifications to Dagre

To implement the functionality of the core path I modified the vertical alignment algorithm as shown in Figure 5.14. Lines highlighted in green are new, the one in orange is modified. The code from line 1-15 is unchanged but not shown here due to space limitations.

The way these changes work is by finding the core node in the current layer and aligning it to the core node from the previous layer, before looking at other nodes. Some cleanup is then needed for the algorithm to continue as before.

Lines 21-27 serve the purpose of altering the current layer by identifying the core node (line 21-23) and creating a new layer with it at index 0 (line 24-27), ensuring it is aligned first. It is possible that no core path is defined so there is a condition to make sure a core node was found before using the *coreNodeIndex*.

For each node its neighbors are saved to the variable *ws*. If the current node *v* is part of the core path (line 31), *ws* is filtered to only contain the one core path neighbor. With this modification a core node can only be aligned to other core nodes.

The only condition that can prevent two core nodes from being aligned is in line 41, when their connecting edge is part of a conflict (edge crossing with at least one inner segment). For the other conditions:

align[v] === *v* is only ever false if *v* has two median neighbors, and we are currently looking at the second neighbor and *v* was aligned to the first one. Since a core node will only ever have one neighbor in *ws*, this condition will always be true.

prevIdx < pos[w] is always true for the very first node in a layer, because *prevIdx* will be -1.

The new condition *pos[w] !== alreadyAlignedIndex* is to prevent other nodes from also aligning to a core node. *alreadyAlignedIndex* is -1 in the first inner iteration and thus this condition is also met.

After a core node was aligned, the modified code checks if it was moved to the front of the layer (line 45) via the boolean *hasShifted*. This variable has been initialized as false (line 19) and potentially set to true in line 26. If *hasShifted* is true the previous index is reset to -1, to not interfere with future alignments. For example, the core nodes could have been both on position 4 in their respective layer, preventing alignments to be made between nodes in positions 0-3. Resetting the previous index to -1 makes those alignments possible again.

alreadyAlignedIndex is set to the position of the neighboring core node, preventing other nodes from being aligned to that same one (line 40). Finally, *hasShifted* is set to false because there can be only one core node for which these changes are made.

The logic in the loop over the nodes on line 28 is changed significantly for the first iteration (the core node) but is nearly the same as the original code from the second iteration onward. Line 40 is the single difference for those later iterations.

5. Modifications to Dagre

```
16 layering.forEach(layer => {
17   let prevIdx = -1;
18   let alreadyAlignedIndex = -1;
19   let hasShifted = false;
20   let modifiedLayer = layer;
21   let coreNodeIndex = layer.findIndex(v => {
22     return isInCorePath(v);
23   });
24   if (coreNodeIndex > 0) {
25     modifiedLayer = [layer[coreNodeIndex], ...layer.slice(0,
26       coreNodeIndex), ...layer.slice(coreNodeIndex + 1)];
27     hasShifted = true;
28   }
29   modifiedLayer.forEach(v => {
30     let ws = neighborFn(v);
31     if (ws.length) {
32       if (coreProcess[v]) {
33         ws = ws.filter((w) => isInCorePath(w));
34       }
35       ws = ws.sort((a, b) => pos[a] - pos[b]);
36       let mp = (ws.length - 1) / 2;
37       for (let i = Math.floor(mp), il = Math.ceil(mp); i <= il; ++i) {
38         let w = ws[i];
39         if (align[v] === v &&
40           prevIdx < pos[w] &&
41           pos[w] !== alreadyAlignedIndex &&
42           !hasConflict(conflicts, v, w)) {
43           align[w] = v;
44           align[v] = root[v] = root[w];
45           prevIdx = pos[w];
46           if (hasShifted) {
47             prevIdx = -1;
48             alreadyAlignedIndex = pos[w];
49             hasShifted = false;
50           }
51         }
52       }
53     }
54   });
55   return { root: root, align: align };
56 } }
```

Figure 5.14.: Modified implementation of the vertical alignment step with core path.

6. Evaluation

Dynamic Dagre is the result of all the modifications presented in the previous chapter. Several aspects of the resulting layouts have been improved and will be quantitatively evaluated in this chapter.

The layouts *Dynamic Dagre* creates are evaluated based on two metrics:

Visual stability in dynamic graph drawing is a measure describing how consistently the positions of nodes and edges are preserved across consecutive steps, relative to the underlying structural changes of the graph.

Layout quality is the extent to which a graph's visual representation satisfies established aesthetic and structural criteria.

The original, unmodified Dagre algorithm, referred to as *Base Dagre*, and the Process Explorer visualization provided by *Celonis*, referred to as *Celonis*, [Cel] will be compared to *Dynamic Dagre*. The comparison to *Base Dagre* is made to quantify the impact of the modifications. *Celonis* is a process mining tool for process exploration, thus having the same goals as *Dynamic Dagre*.

A handful of other algorithms were considered:

Dynagraph [dyn] provides functionality for dynamic layout creation. However, it no longer appears to be actively maintained. The GitHub repository lacks documentation, the official website is outdated, and the most recent release dates back to 2006. Installation attempts were unsuccessful, suggesting that the tool is not suitable for practical or commercial use.

yFiles [yWo] offers incremental layout with the goal to preserve the mental map, but is only accessible through a subscription.

Pro M Tools[Pro] is a large collection of process mining tools including process visualizations. The functionalities are provided by plug-ins with a default collection of them recommended by the maintainers. There seems to be no plug-in dedicated to dynamic process visualization. Furthermore, most of their graph visualizations are based on Petri nets, whereas *Celonis* and the tool I used to visualize *Dynamic Dagre* layouts employ BPMN notation.

Disco by Fluxicon [Flu] is another tool for process visualization that uses BPMN notation. Unfortunately the data set I used was too large for that tool.

The BPI Challenge 2019 dataset was used for both development and evaluation. It is a publicly available event log provided by the IEEE Task Force on Process Mining [IEE]. Since only a limited number of high-quality event logs are publicly accessible, this dataset is commonly employed in process mining research. This data set contains 252 thousand cases forming roughly 8500 variants with over 1.5 million events relating to 42 distinct activities performed.

6.1. Methodology

To compare the three layout creation algorithms *Base Dagre*, *Dynamic Dagre*, and *Celonis*, I visualized the six most common variants in the BPI Challenge 2019 dataset with each of these algorithms and analyzed the results. First, only the single most common variant was laid out, next the two most common variants, and so on, until the six most common variants were included. The fifth most common variant adds only one edge to the graph and was not considered in isolation, but it is included in the final layout. The resulting graph conditions are: variant 1, variants 1–2, variants 1–3, variants 1–4, and variants 1–6.

When creating layouts, *Dynamic Dagre* only considers the immediate predecessor, not earlier layouts. This means that the same graph can receive different layouts depending on the sequence of preceding versions. For example, a graph may be explored by stepping forward, backward, or skipping intermediate versions, and the resulting layout can vary accordingly. In practice, the most stable results are obtained when the layout is built incrementally, one by one, starting from a single variant and extending it with additional variants. In some situations, it can even be beneficial to lay out the same graph multiple times so that the algorithm can refine node positions and adjust edge routing based on prior knowledge of the layout state.

Both versions of Dagre were used inside the Process Analyzer tool by process.science, where I had access to modify the Dagre implementation [pro]. This also allowed me to add debugging statements to determine the exact positions of nodes and edges.

Celonis layouts were created in their online tool Process Explorer with an academic license. Without access to the source code I relied on visual analysis of the resulting layouts. This tool also uses a different way to add variants to the graph: here you add activities (nodes) and connections (edges) individually instead of adding a whole variant. This limited the amounts of identical graphs, between the three algorithms, I was able to create and is the reason only five graph versions are compared.

To calculate the positions and angles of nodes and edges I took screenshots of the five graph versions, scaled them uniformly and measured the pixel distances. This process lead to a slight inaccuracy in the results, estimated to be up to 5% of variance from the actual values.

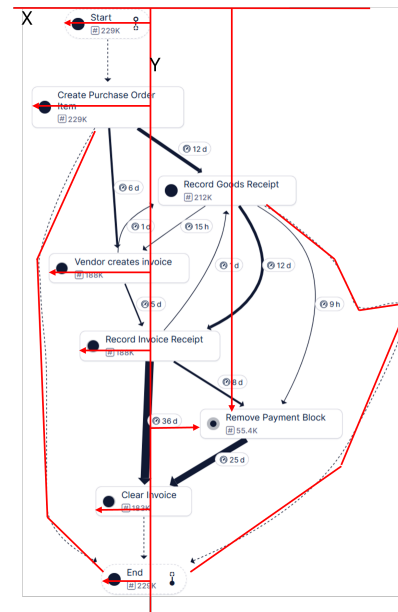


Figure 6.1.: Visual analysis of a Celonis layout.

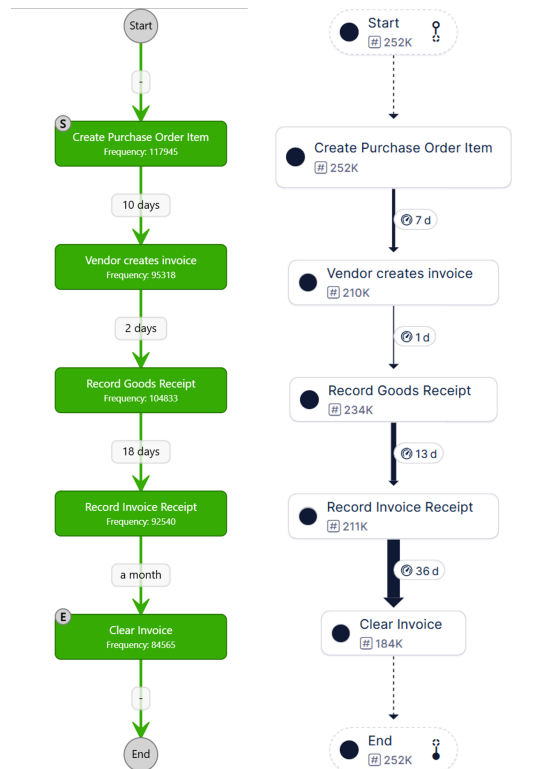
6. Evaluation

In Figure 6.1, the graph in black is a screenshot of *Celonis*, the lines in red are drawn in to measure positions. This image only shows some exemplary lines, in practice all nodes and edges were measured in this manner. This was done in Microsoft Power Point due to a number of useful functionalities: Turning the screenshots transparent and scaling them while preserving aspect ratio allowed larger graphs to be scaled the same as smaller ones. Lines drawn over the graph have exact measures that can be read easily. The ruler tool allows to measure angles conveniently.

The vertical red line in the middle is the visual center of the tool in which these graphs were laid out, it serves as a reference for X coordinates between layouts. From there horizontal arrow headed lines measure the X coordinates of the nodes.

The horizontal line at the top is the X axis from which Y coordinates are measured. One exemplary arrow measures the Y coordinate for the node "Remove Payment Block". Node positions were calculated as the top left corner ignoring rounded edges, the same way Dagre defines its positions.

Edge lengths were measured with one or multiple line segments, as can be seen on the very left and very right in Figure 6.1. Edges in Dagre have a property *points*, an array of positions where its dummy nodes were, which was used for measuring their length. The pairwise distance from the first to second point, second to third and so on were added to calculate the total length. Approximating *Celonis*' edges with straight line segments mirrors that method. Straight line segments were also used to calculate angles between edges, although those segments were shorter and more precisely aligned to the edges orientation.



Coordinates and lengths in *Celonis* layouts were scaled to match Dagres. The very first layout is practically identical for all algorithms as can be seen in Figure 6.2, but the calculated positions of *Dagre* didn't match the measured distances from *Celonis*. In this example the Y coordinate of node "End" is 650 for both *Dagre* versions but 1524 for *Celonis*. This ratio r is used to scale *Celonis* positions and length: $r = \frac{650}{1524} = 0.4265$.

In both *Dagre*'s layouts the nodes are colored either green or red, indicating whether they are part of the target process or not. Edges colors indicate whether that specific transition is in the target process.

Dynamic Dagre uses the path in Figure 6.2 as the *core path*.

Figure 6.2.: Dagres and Celonis layout for the most common variant. 37

6.2. Visual Stability

Lezama et al. propose nine metrics for quantifying visual stability between two graph layouts [Lez+15]. Among these, the **graph drawing offset (GDO)** is particularly relevant. It measures changes in node positions by computing the Euclidean distance between the positions of each node present in both layouts and summing these distances across all nodes.

In this work, only the GDO is employed to compare visual stability, as Lezama et al. identified it as one of the most influential metrics, while the others are either not applicable for the present comparisons or do not add information.

Vertex set stability (VS) measures the percentage of nodes that are present in both layouts. Since this analysis compares different algorithms applied to the same underlying graph, all algorithms yield identical VS values. This metric is therefore not included.

Edge set stability (ES) is defined analogously for edges and is excluded for the same reason.

Vertex degree change (VDC) quantifies differences in the degree of each node between two layouts. Like the set stability measures, it depends solely on the underlying graph structure rather than the layout, and is thus not applicable in this context.

Vertex drawing active positions (VDAP) and **Edge drawing active positions (EDAP)** assume a global layout, as might be produced by an offline algorithm. For online algorithms, their values are always equal to 1 by definition. Since both *Dagre* variants are online algorithms and *Celonis* also appears to be online, these two measures add no information.

Three of the remaining measures rely on so-called logical positions, which are only briefly explained in the original paper. The authors assume that algorithms compute *vertex logical positions* and *edge logical positions*, which are then translated into coordinates in Euclidean space. The combination of rank and order of a hierarchical layout could be interpreted as such logical positions. However, there are no distinct sets for vertex and edge positions. If rank and order combinations are taken as the logical positions of nodes and edges, several problems arise.

Consider a layout L_O and its mirrored version L_M . Suppose the next graph version introduces an edge from *Start* to *End*. In L_O , the new edge is placed on the far left while the rest of the layout remains unchanged. In L_M , the new edge is placed on the far right, again resulting in a mirrored version of L_O . Under the rank–order interpretation, nearly all nodes and edges in L_O would change their logical positions, with only *Start* and *End* remaining stable. In L_M , by contrast, no logical positions would change at all.

This discrepancy demonstrates that the measure is not invariant under mirroring and, more generally, that it is highly dependent on how logical positions are defined. Since mirrored layouts are equally readable to humans, this definition of logical positions is considered inapplicable. As no alternative definition could be identified that would be meaningful in the context of these algorithms, the three measures based on logical positions are excluded from the evaluation.

6. Evaluation

Although the GDO only accounts for node positions and does not directly measure edge stability, this limitation is not critical in practice. Edges naturally move with their incident nodes, and testing has shown that significant changes in edge placement occur only when their corresponding nodes change position. Both *Dynamic Dagre* and *Celonis* produce layouts that are sufficiently stable in this regard. Consequently, empirical testing indicates that GDO values quantify visual changes effectively and serve as a suitable measure of visual stability in this evaluation.

GDO	Base Dagre	Dynamic Dagre	Celonis
1	-	-	-
2	388.7	0	820
3	101.5	0	195.7
4	1539.2	0	1678.9
6	2490.8	369.7	765.1

Table 6.1.: Graph drawing offset for the three algorithms on six versions of a dynamic graph.

Table 6.1 presents the GDO values for the three algorithms. Each row shows the GDO relative to the previous graph version. The first row is empty because it corresponds to the base version. For example, row 4 reports the GDO from the third to the fourth graph version. Small GDO values indicate few node movements, with a value of 0 meaning that only new elements were added and no existing nodes moved. High GDO values indicate substantial node movement, either due to a few nodes moving large distances or many nodes moving.

The first layout in Figure 6.2 showing only one case is identical between the two *Dagre* versions, and nearly identical to the layout created by *Celonis*. Figures 6.3 through Figure 6.6 show each of the algorithms layouts for the other versions, including increasingly more process variants.

Since there is no previous layout to compare against, the first row of GDO values is empty.

In the second row, *Dynamic Dagre* has a GDO of 0, indicating that none of the existing nodes were repositioned and that the new edges were integrated without altering the previous layout. In contrast, both *Base Dagre* and *Celonis* reassigned node positions, most notably by swapping the ranks of *Vendor creates invoice* and *Record Goods Receipt*. The *Celonis* layout also expands considerably in width, resulting in a GDO of 820. *Base Dagre*, by contrast, moves very little aside from the rank swap, yielding a relatively small GDO of 388.7. This movement negatively affects visual stability and disrupts the user’s mental map: it increases the overall displacement between layouts and alters the relative positioning of activities. Because of the symmetry of their incident edges, *Vendor creates invoice* and *Record Goods Receipt* could theoretically exchange labels without invalidating the structure (except for edge values). The underlying data appears to favor this ranking, leading both algorithms to disturb the established flow. In contrast, *Dynamic Dagre* preserves the ranks and maintains the central alignment of the core path.

6. Evaluation

In the third graph, the layouts show the least amount of movement between conditions, as indicated by the low GDO values. The lowest for each algorithm respectively. This is expected given the minimal change in the underlying graph: only one new edge is added, connecting *Record Goods Receipt* to *End*.

For both *Base Dagre* and *Celonis*, this addition shifts *Record Goods Receipt* outward and moves *End* in the same direction. In addition, *Celonis* repositions several nodes further to the right to accommodate the new edge.

By contrast, *Dynamic Dagre* again integrates the edge without relocating existing elements. In both *Base Dagre* and *Celonis*, however, *Vendor creates invoice* and *Record Goods Receipt* maintain the relative positions introduced in the previous layout, even though reverting to the initial ranking would have produced a shorter new edge.

The fourth graph introduces the new node *Remove Payment Block* along with three new edges, creating both a new cycle and a branching path from *Record Goods Receipt* to *Clear Invoice*.

Base Dagre changes its layout substantially, to the point of being nearly unrecognizable. Evidently, the DFS used to identify cycles started at *Record Invoice Receipt*. The DFS moved through *Record Goods Receipt* to *Vendor creates invoice* and then flagged the edge connecting that activity back to *Record Invoice Receipt* as an edge to reverse. Similarly, the edge from *Record Goods Receipt* to *Record Invoice Receipt* is discovered after its antiparallel counterpart and is therefore also flagged for reversal. With these edges internally reversed, *Record Invoice Receipt* no longer has incoming edges and is positioned near the top, potentially giving users the false impression that it is a source node.

Dynamic Dagre does not exhibit this issue. It preserves the existing ranking and integrates the new elements without relocating nodes, scoring a GDO of 0 once more. However, the edge connecting *Record Goods Receipt* to *End* is moved to the right to add the new node where this edge was positioned previously.

Celonis maintains the relative ranking of nodes but introduces substantial horizontal movement, with nearly all nodes shifting noticeably. This results in the highest GDO observed for this algorithm 1678.9, more than twice the value of its next highest GDO. The three nodes *Vendor creates invoice*, *Record Invoice Receipt*, and *Clear Invoice* preserve their relative positions to one another, but all shift leftward. Similarly, *Start* and *Create Purchase Order* move together to the left. The remaining nodes, including the new one, are repositioned further to the right. This layout also contains a preventable edge crossing, which could be resolved by swapping the positions of *Vendor creates invoice* and *Record Goods Receipt*. The movement, and especially the changes in relative positioning, damage the user's mental map and hinder their ability to follow the evolution of the process model.

6. Evaluation

For the last graph version, only two edges are added, but the one connecting *Record Invoice Receipt* with *Remove Payment Block* forces these two nodes onto separate ranks.

Base Dagre undergoes pronounced changes once more. The new layout shows the same issue of starting the DFS for cycle removal from *Record Invoice Receipt*. Based on the node positions, the layout appears to be mirrored relative to the previous version and then modified to include the new edges. This suggests that the sweeps to identified different optimal orders were in different directions for these layouts, because the direction influences how the bias is interpreted. What is internally a left bias, looks like a right bias in the resulting layout to users. Further inspection shows that the edge crossings occur on higher ranks in the fourth layout and on lower ranks in the fifth, supporting this interpretation. The result is the highest recorded GDO in this thesis, 2490.8, which highlights the very low visual stability of this algorithm.

Celonis maintains most of the relative positioning. *Record Invoice Receipt* is moved to the right of *Vendor creates invoice* instead of remaining to the left. *Remove Payment Block* is necessarily moved to a lower rank, and the new edge to *End* shifts that node further left. Together, these displacements result in a GDO of 765.1, comparable to that of the second graph version. The previous edge crossing is resolved by placing the antiparallel edges next to one another, but a new crossing appears above *Vendor creates invoice*.

The introduction of a new rank forces a GDO greater than 0 for *Dynamic Dagre*. Nevertheless, the value of 369.7 remains the lowest, apart from the third graph version and about half of *Celonis*'s value for this condition. *Remove Payment Block*, *Clear Invoice*, and *End* are each shifted one rank lower. With no other node remaining on its rank, *Remove Payment Block* can be positioned closer to the core path and is placed further to the left.

Overall, *Dynamic Dagre* demonstrates superior visual stability compared to both *Base Dagre* and *Celonis*. It preserves node ranks and relative positioning across incremental graph versions and integrates new elements without disrupting existing layouts. As a result, it achieves the lowest GDO values in all scenarios, reflecting minimal movement of nodes and strong preservation of the mental map. In contrast, *Base Dagre* and *Celonis* frequently reassign node positions and introduce unnecessary displacement, which reduces stability and increases the cognitive load on users. These results indicate that *Dynamic Dagre* is particularly well-suited for dynamic graph visualization scenarios where maintaining continuity and readability across versions is critical.

An additional benefit of *Dynamic Dagre* is observed when the graph is consistently aligned so that the core path remains in the same position on the screen. In this configuration, changes between versions become immediately apparent to the user, while unchanged elements are preserved as pixel-perfect copies. This property further enhances visual stability and supports the user's ability to track modifications across consecutive layouts.

6. Evaluation

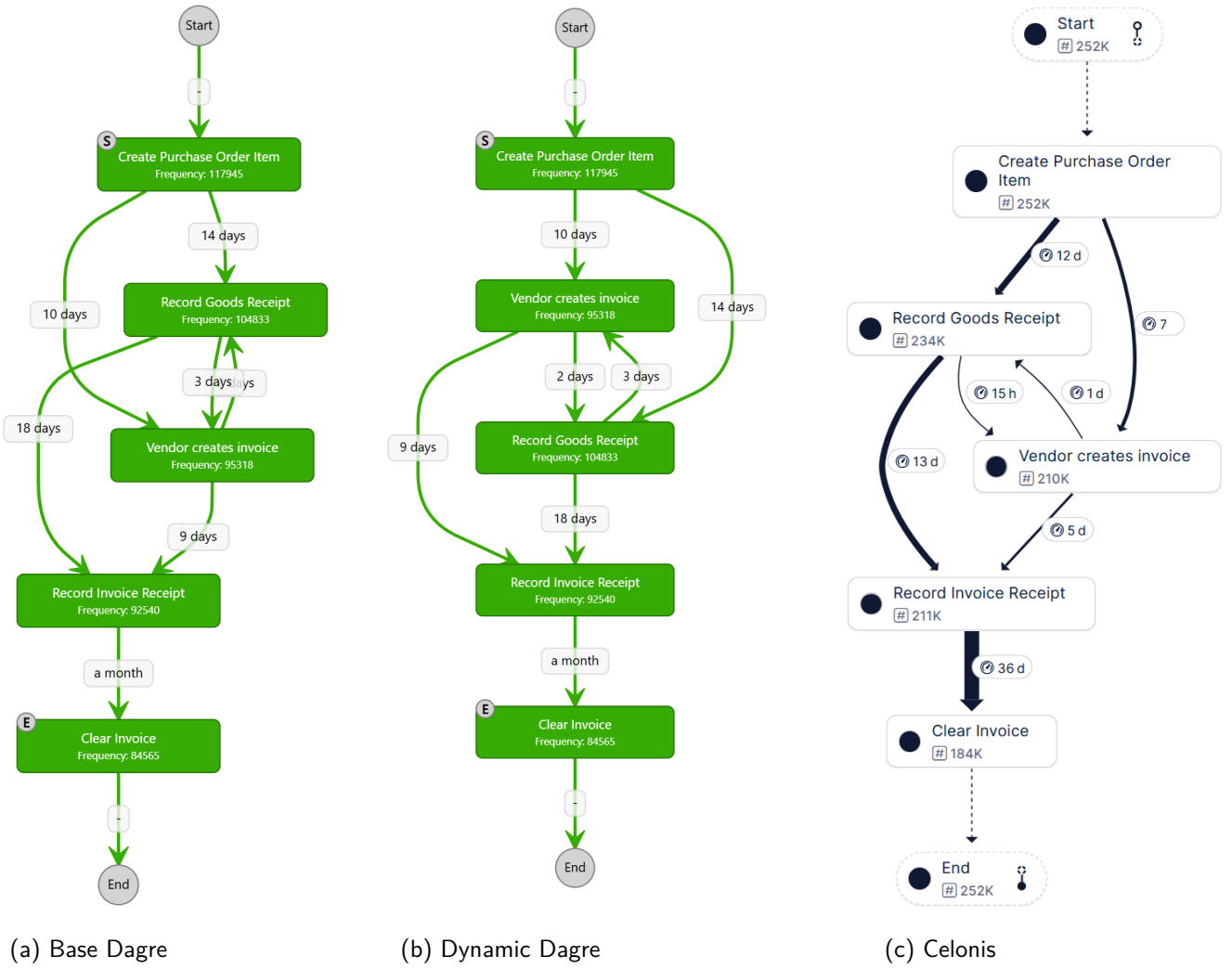


Figure 6.3.: Second layout for each algorithm.

6. Evaluation

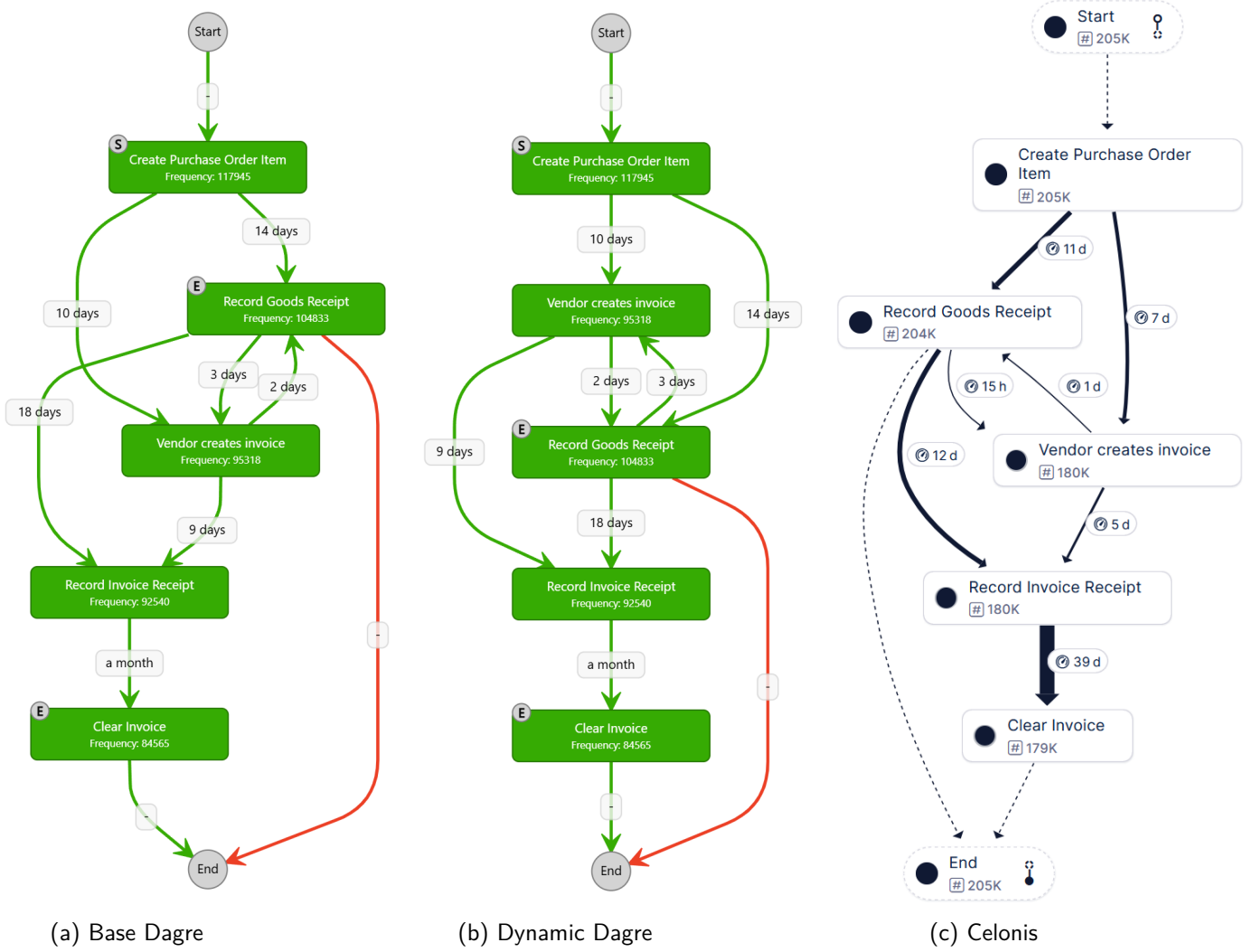
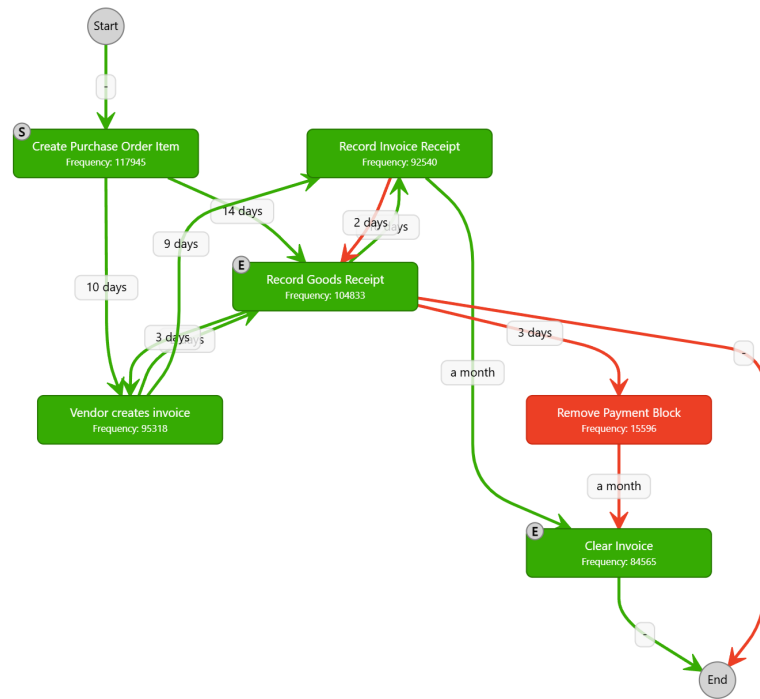
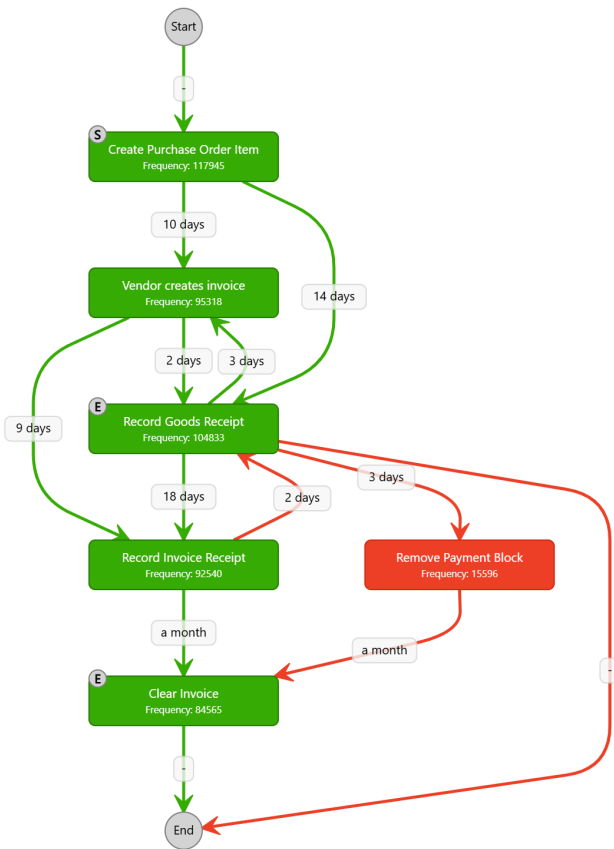


Figure 6.4.: Third layout for each algorithm.

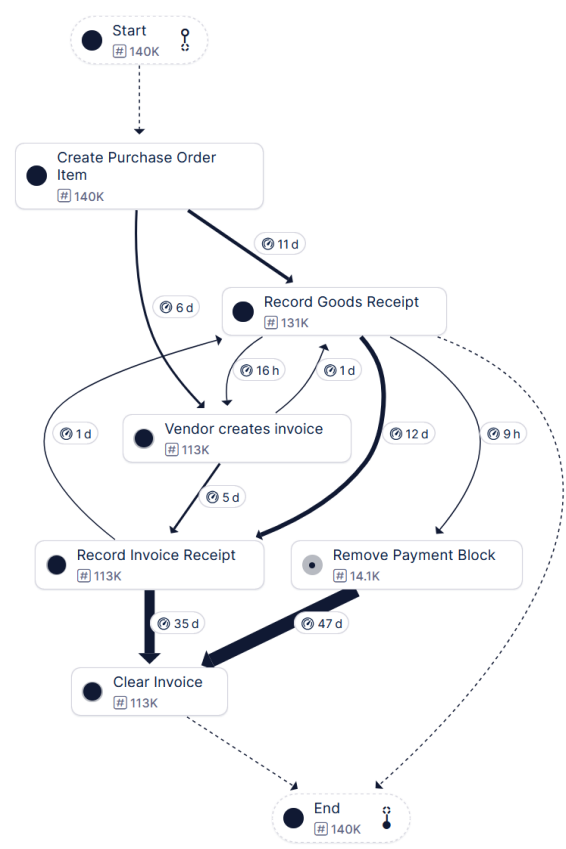
6. Evaluation



(a) Base Dagne



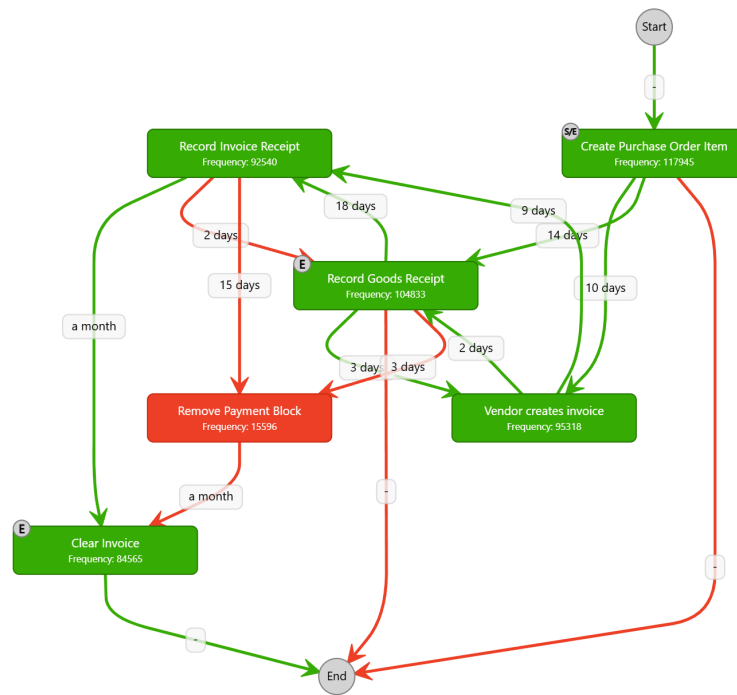
(b) Dynamic Dagne



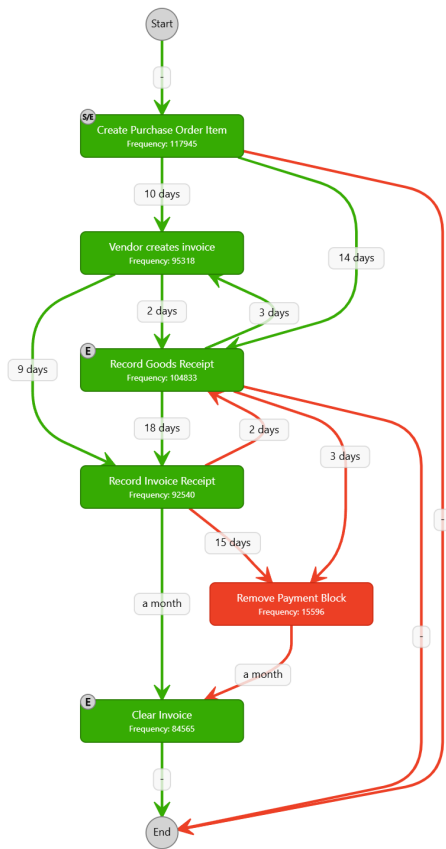
(c) Celonis

Figure 6.5.: Fourth layout for each algorithm.

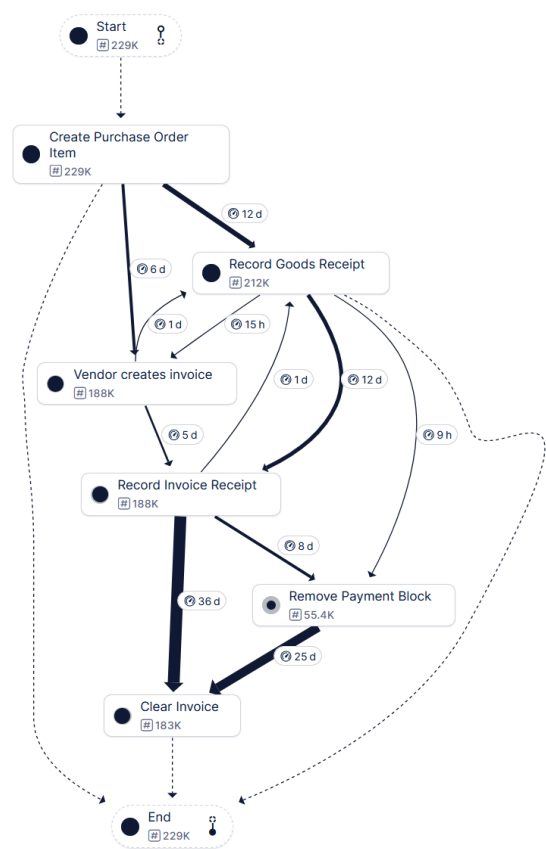
6. Evaluation



(a) Base Dagre



(b) Dynamic Dagre



(c) Celonis

Figure 6.6.: Fifth layout for each algorithm.

6.3. Layout Quality

While visual stability is a crucial metric for dynamic layouts, it does not provide information about the quality of a single layout in isolation. The goal of *Dynamic Dagre* is to support dynamic layouts without compromising overall layout quality. In this section, *Dynamic Dagre* is compared to *Base Dagre* and *Celonis* with respect to layout quality.

The evaluation criteria are taken from "An Aggregation-Based Overall Quality Measurement for Visualization" by Huang et al. [Hua13]. They propose four key criteria for assessing graph layouts:

1. Minimize the number of edge crossings (cross#)
2. Maximize crossing angle resolution (crossRes)
3. Maximize node angular resolution (angularRes)
4. Uniformize edge lengths (uniEdge)

Each of these factors addresses a specific challenge in graph readability.

A high number of edge crossings quickly clutters a drawing and makes it difficult to trace paths, so layouts with fewer crossings are generally preferred.

When crossings do occur, they should ideally meet at large angles, since shallow angles are visually confusing. This is captured by the crossing angle resolution.

Similarly, node angular resolution considers the minimum angle between edges incident to the same node: layouts with larger angles make it easier to distinguish different connections, whereas small angles can cause edges to overlap visually.

Finally, uniform edge lengths contribute to a balanced appearance and prevent distortions in perception, as very short or very long edges may suggest artificial emphasis in parts of the graph. The value for uniform edge lengths is the standard deviation of edge lengths, where smaller values indicate greater uniformity.

The methodology does not yield absolute values but instead enables comparison between layouts by calculating z-scores for each criterion, which express how many standard deviations a value lies above or below the mean of all layouts [Her07]. The z-scores for the four categories are then aggregated into an overall quality score (O) as follows:

$$O = -z_{\text{cross\#}} + z_{\text{crossRes}} + z_{\text{angularRes}} - z_{\text{uniEdge}}$$

The best layout is the one with the highest O value.

One limitation of this measure is that it assumes a strictly linear relationship between the four criteria and overall layout quality. In practice, the effect of some criteria can be more complex: for example, the authors themselves report evidence of a quadratic relationship between crossing angle resolution and task performance. Nevertheless, the aggregated score O provides a practical and interpretable way to compare layouts across multiple dimensions, and thus serves as a suitable measure for this evaluation.

6. Evaluation

$z_{\text{cross\#}}$	Base Dagre	Dynamic Dagre	Celonis
1	-	-	-
2	1.41	-0.71	-0.71
3	1.41	-0.71	-0.71
4	1.39	-0.93	-0.46
6	1.4	-0.89	-0.51

Table 6.2.: Z-scores for the amount of edge crossings (cross#). Smaller values are better and highlighted in green. Larger values are red.

Comparing the **number of edge crossings** is relatively straightforward. For the first layout, none of the algorithms produce crossings. In the second and third layouts, however, only *Base Dagre* includes exactly one crossing each. The fourth layout generated by *Base Dagre* performs considerably worse, introducing five crossings due to its rearranged and convoluted structure. *Celonis* also produces one crossing in this layout, in a similar position to the earlier crossings of *Base Dagre*.

In the final layout, *Celonis* resolves the earlier antiparallel crossing between *Record Goods Receipt* and *Record Invoice Receipt* by placing the edges next to each other. However, a new crossing is introduced above the node *Vendor creates invoice*, where the edge to *Record Goods Receipt* overlaps slightly with the incoming edge from *Create Purchase Order Item*. Although there is sufficient space to separate these edges, the overlap may have been too small to be detected as a crossing internally. In contrast, *Base Dagre* once again restructures the entire layout, this time resulting in six crossings overall.

The resulting z-scores are shown in Table 6.2. *Dynamic Dagre* consistently achieves the best performance with zero crossings in all layouts. *Celonis* performs comparably up to the fourth layout but introduces one crossing each in the final two cases. *Base Dagre* performs worst across all versions, with crossings in every layout beyond the first.

z_{crossRes}	Base Dagre	Dynamic Dagre	Celonis
1	-	-	-
2	-	-	-
3	-	-	-
4	-1.37	0.99	0.38
6	-0.32	1.35	-1.04

Table 6.3.: Z-scores for the minimum angle of edge crossings (crossRes). Larger values are better and highlighted in green. Lower values are red.

Measuring **crossing angles** was less straightforward than counting crossings. The angles were determined manually by analyzing screenshots of the layouts and measuring the degrees at each crossing point. While this introduces minor inaccuracies, the differences between algorithms were large enough that these did not meaningfully affect the results.

6. Evaluation

For comparability, layouts without crossings were assigned the optimal angle of 90° . This ensures that all algorithms can be evaluated consistently, but it also means that when all algorithms achieve the same optimal value, the standard deviation becomes 0 and z-scores cannot be calculated.

In layouts 2 and 3, this situation occurs: *Base Dage* has a single crossing with an angle of 90° , while *Dynamic Dage* and *Celonis* have no crossings and are assigned the optimal value. Since all values are identical, no z-scores are available for these layouts.

In the fourth layout, *Base Dage* produced several crossings, with the smallest angle measured at 40° . *Celonis* had a single crossing with a more favorable angle of 77° . *Dynamic Dage*, having no crossings, was again assigned the optimal angle of 90° .

In the fifth layout, *Base Dage* again introduced multiple crossings, this time with a narrowest angle of 32° . *Celonis* performed particularly poorly in this case, producing a crossing with the very small angle of 7° , resulting in its lowest score overall.

Although this metric provides relatively few measurable values, the available results consistently favor *Dynamic Dage*, which maintains an optimal crossing angle by avoiding crossings entirely. *Base Dage*, despite generating more crossings, tends to maintain reasonably wide angles. By contrast, *Celonis* exhibits a severe outlier in the final layout, with a near-parallel crossing that substantially reduces its performance.

$z_{\text{angularRes}}$	Base Dage	Dynamic Dage	Celonis
1	-	-	-
2	-0.31	-1.04	1.35
3	1.37	-0.37	-0.99
4	-0.82	-0.59	1.41
6	1.07	-1.34	0.27

Table 6.4.: Z-scores for the minimum angle of edges incident to the same node (angularRes). Larger values are better and highlighted in green. Lower values are red.

Like before, the **angular resolutions** were determined manually by measuring screenshots, which may introduce slight inaccuracies. However, in contrast to crossing angles, many of the measured values here are very close to one another: in five cases, pairs of angles differed by less than 5° . At the same time, each graph version showed at least one algorithm with a noticeably higher or lower angular resolution than the others, resulting in a distinct best or worst performer in every condition.

It is worth noting that the original study by Huang et al. used graphs where nodes were represented as dots and edges were drawn directly toward their centers. In the present graphs, the nodes are considerably larger, giving incident edges more space between them and thereby reducing the visual impact of small angle differences. For this reason, overall scores excluding angular resolution are discussed separately later.

6. Evaluation

The first row contains no values, as all angles in all layouts were 180° , resulting in identical outcomes and thus no z-scores.

For the second layout, the smallest measured angles were as follows:

Base Dagre produced 20° for the pair of antiparallel edges. *Dynamic Dagre* produced 16° between the two rightmost edges incident on *Record Goods Receipt*, though with noticeably more spacing than in *Base Dagre*. *Celonis* achieved the widest angles, with its smallest measured value being 29° for the outgoing edges from *Record Goods Receipt*.

For the third layout, *Base Dagre* widens its smallest angle to 30° , twice as large as the corresponding values for the other algorithms. *Dynamic Dagre* simply adds the new edge without repositioning existing nodes, resulting in the same 16° as before. In contrast, the outgoing edges from *Record Goods Receipt* in *Celonis* become increasingly crowded, with the smallest angle reduced to 11° .

In the fourth graph version, *Base Dagre* produces a substantially altered layout with multiple very narrow and tightly packed edges, the smallest of which was measured at only 2° . *Dynamic Dagre* places its new edges close to one another and to the existing red edge, resulting in a minimal angle of 4° . Although this is still a small value, the additional spacing makes the edges visually more distinct than in *Base Dagre*. *Celonis* creates a wider layout in this step, leaving more room for edges, and achieves an angular resolution of 22° .

The final layout of *Base Dagre* untangles somewhat and reaches an angular resolution of 10° , the best value for this graph version. *Dynamic Dagre*, however, contains two long incoming edges toward *End* that are nearly parallel, separated by only 1° . The edge crossing in the last layout of *Celonis* occurs very close to a node, resulting in an angular resolution of 7° . In addition, these edges overlap without any visible whitespace in between.

Overall, there is no consistent best-performing algorithm for angular resolution. *Dynamic Dagre* stands out with the worst average score, as it prioritizes stability and does not expand the layout to create additional space for edges. The other two algorithms alternate between achieving the best angular resolution across different graph versions, but each have the worst at one point too. As noted earlier, due to concerns regarding the applicability of this measure to the current setting, results excluding angular resolution are discussed later.

6. Evaluation

z_{uniEdge}	Base Dagre	Dynamic Dagre	Celonis
1	-	-	-
2	0.47	0.92	-1.39
3	1.22	0.01	-1.23
4	-0.45	1.39	-0.93
6	-1.29	1.14	0.15

Table 6.5.: Z-scores for the standard deviation of edge lengths (uniEdge). Smaller values are better and highlighted in green. Larger values are red.

Uniform edge lengths are inherently difficult to achieve in hierarchical layouts, as some edges necessarily span multiple ranks while others connect only neighboring ranks. Since all three algorithms employ a hierarchical approach, none can fully eliminate this limitation.

In the first layout, all algorithms produce practically identical drawings. The standard deviation in edge length is zero across the board.

From the second layout onward, however, clear tendencies emerge. *Dynamic Dagre* preserves its layout so faithfully that new edges are simply added to the sides rarely repositioning existing elements. In contrast, both *Base Dagre* and *Celonis* adjust their node positions to accommodate changes. Of these, *Celonis* generally manages this better, producing the lowest deviation and thus the best z-scores in most cases.

In the third graph version, both *Base Dagre* and *Celonis* lose some of their advantage by placing *Record Goods Receipt* unnecessarily high, thereby elongating its edge to *End*. Even so, *Celonis* still achieves the best score, with *Dynamic Dagre* following in second place.

The wider layout of *Celonis* in the fourth graph version proves beneficial for uniform edge lengths, as it balances rank-spanning and shorter edges. *Base Dagre* has one rank less than the other algorithms and thus improves uniformity here, with its score of 134.2 only slightly worse than the 117.5 achieved by *Celonis*. *Dynamic Dagre*, by contrast, suffers: its already longest edge is stretched further outward by the new elements, producing a deviation of 198.6, by far the worst result so far.

In the final graph version, *Base Dagre* reduces the number of ranks compared to the other algorithms again, having 2 less. This choice significantly improves its uniformity score, bringing the standard deviation down to 145.6, the best result for this graph version. *Celonis* reaches second place with 213.9, although a small kink in one of its long edges worsens its result slightly. *Dynamic Dagre*, however, performs worst once more with 260.4. Simply routing the new long edge to the left, where fewer other elements interfere, might have been enough to close the gap to *Celonis*.

Celonis generally performs best for uniform edge lengths, suggesting that this criterion is explicitly considered in its layout strategy. *Base Dagre* only overtakes in the final graph, exploiting its inconsistent rank structure. *Dynamic Dagre*, by design, performs worst: the emphasis on preserving stability and minimizing change comes at the cost of longer and less uniform edge lengths.

6. Evaluation

O	Base Dagre	Dynamic Dagre	Celonis
1	0.0	0.0	0.0
2	-2.19	-1.26	3.45
3	-1.27	0.32	0.94
4	-3.12	-0.06	3.18
6	0.65	-0.24	-0.41

(a) Overall quality score O .

O no angularRes	Base Dagre	Dynamic Dagre	Celonis
1	0.0	0.0	0.0
2	-1.88	-0.21	2.10
3	-2.63	0.70	1.94
4	-2.30	0.53	1.77
6	-0.42	1.10	-0.68

(b) O without angular resolution.

Table 6.6.: Overall quality score O for all algorithms with and without angular resolution. Larger values are better and highlighted in green. Lower values are red.

Looking at the aggregated results in Table 6.6 a, *Celonis* achieves the best overall scores for all but the final graph version, where *Base Dagre* unexpectedly takes the lead. In the earlier layouts, *Celonis* maintains a significant margin over both competitors, while *Dynamic Dagre* consistently occupies second place. *Base Dagre*, by contrast, performs poorly overall despite its late surge.

When angular resolution is excluded (Table 6.6 b), the relative order remains unchanged for the first four layouts, though differences become more pronounced in the third. The final layout, however, shifts markedly: *Dynamic Dagre* now comes out on top, while *Celonis* drops to last place.

The breakdown of individual measures clarifies these results. *Dynamic Dagre* excels in minimizing edge crossings and thus maximizing crossing resolution, criteria that Huang et al. highlight as particularly impactful for perceived layout quality. Its rigidity, however, results in poor performance on angular resolution and uniform edge lengths.

Base Dagre, although generally the weakest algorithm, can sometimes benefit from its fragmented rank structure, which artificially improves its score despite producing unintuitive and less readable layouts.

Celonis, in turn, dominates most scenarios by balancing edge uniformity and angular resolution, but loses ground in the final layout due to a combination of a narrow angle crossing and weaker edge length uniformity.

If *Dynamic Dagre* and *Celonis* are compared in a head-to-head evaluation, excluding *Base Dagre* from the z-score calculations, the results become more direct. In this setting, *Celonis* performs better in the first two layouts, while they tie in the final two. When angular resolution is excluded, *Dynamic Dagre* outperforms *Celonis* in the last two graph versions.

Taken together, these results suggest that *Dynamic Dagre* offers a clear improvement over *Base Dagre* in terms of layout quality. While it does not yet match the sophistication of *Celonis* in most cases, its exceptional handling of edge crossings, a factor arguably underweighted in the aggregated measure, remains a decisive strength.

7. Discussion

This thesis presented the *Dynamic Dagre* algorithm, an enhanced version of the established *Dagre* layout algorithm. This approach extends *Dagre* with targeted modifications that improve stability while also introducing new functionality for dynamic graph layouts.

At its core, *Dynamic Dagre* preserves the relative ranks and orders in its hierarchical layouts, thereby maintaining the user's mental map and improving comprehensibility when process models evolve. A further contribution is the concept of a *core path*, which ensures that a designated path, often the target process or the most frequent variant, remains central and undergoes minimal changes as additional variants are added. This feature directly supports process mining tasks such as exploration and conformance checking, where analysts rely on both stability and clarity to detect deviations.

The modifications and their implementations were described in detail and quantitatively analyzed. While the main focus of this thesis lies on visual stability, where *Dynamic Dagre* demonstrates clear strengths, the evaluation also showed that layout quality remains on par with established tools.

7.1. Research Questions

In the introduction, the following research questions (RQ) were defined as the guiding framework for this work:

- RQ1 How can properties of process mining process models be leveraged to develop a dynamic layout creation algorithm?
- RQ2 Can established layout creation algorithms be modified to visualize dynamic process models?
- RQ3 How does the resulting layout quality and stability compare to other algorithms for visualizing process models?

RQ1: How can properties of process mining process models be leveraged to develop a dynamic layout creation algorithm?

Process models are particularly well suited to hierarchical graph layouts because they exhibit an inherent flow from a unique *Start* event to a unique *End* event. This natural directionality aligns directly with the layered structure of hierarchical layouts, where ranks reflect the temporal order of activities. Preserving such ranks is crucial: the relative vertical positioning of nodes helps analysts quickly interpret the sequence of activities, including loops where processes may cycle back.

Relative ordering within ranks can support users in associating particular branches or behaviors with specific spatial regions of the drawing. However, this spatial mapping is not guaranteed by the model itself, it only becomes meaningful if the layout consistently preserves these orderings across versions. For instance, with a stable layout the right side of the graph might consistently correspond to variants where an *Order Confirmation* occurs at the beginning, allowing analysts to recognize such deviations at a glance.

The central property exploited in this work is the notion of a core path. In process mining, the most frequent variant or a designated target process often serves as the primary reference for analysis. By ensuring that this path remains stable and centrally located, *Dynamic Dagre* creates a reliable anchor for the visualization. With the core path stable, branches and deviations naturally fall into consistent spatial regions around it, making changes immediately visible to the analyst.

RQ2: Can established layout creation algorithms be modified to visualize dynamic process models?

This thesis demonstrates that established layout creation algorithms, specifically Dagre, can be successfully adapted to support dynamic process models. Dagre proved to be a suitable foundation: it is open source, was already integrated in JointJS and thus my work environment, and its modular design allowed for targeted modifications.

Most changes were deliberately kept non-invasive. In many cases, the original algorithm contained decision points that had a significant impact on the resulting layout but were opaque to users. These choices, such as the order of traversals or the selection of edges during cycle removal, were determined by underlying data structures rather than user-defined preferences, and the behavior was thus effectively random or at least unpredictable. By making decisions at these points explicitly oriented towards layout stability, the algorithm was adapted while preserving its original guarantees.

7. Discussion

Some modifications went beyond these minimal interventions but still fit seamlessly into the algorithm’s existing structure. Examples include refinements to the acyclic transformation and the introduction of additional sweeps during order calculations. The most substantial addition is the *core path* concept, which ensures that a designated reference path remains central and stable throughout the layout process. This behavior does not emerge from the original Dagre implementation, but was integrated in a way that respects existing algorithmic constraints while giving the core path preferential treatment.

All new dynamic features were implemented as configurable options that can be disabled in each a Dagre layout call, preserving compatibility with the original algorithm.

The workflow that guided these modifications followed an iterative design approach: identifying problematic behavior (e.g., unstable rankings), suggesting potential fixes, implementing and testing them, and revising the approach where necessary, as described in the procedure model.

RQ3: How does the resulting layout quality and stability compare to other algorithms for visualizing process models?

The evaluation shows that the *Dynamic Dagre* algorithm achieves substantial improvements in layout stability compared to the original *Dagre* implementation, which performed poorly in this respect. *Dynamic Dagre* also significantly outperformed *Celonis*, a commercial process visualization tool.

One of the key strengths is its ability to maintain the *core path* across multiple stages with pixel-perfect consistency. More generally, node stability was quantitatively measured using offset-based metrics (summing the distance of each node between two layout states), where *Dynamic Dagre* consistently produced the lowest values. Although not measured explicitly, edges also exhibited high stability due to the preserved node positions.

In terms of layout quality, results were more mixed. The algorithm performs well in avoiding edge crossings, which is one of the most important readability factors. However, the prioritization of stability sometimes comes at the expense of uniform edge lengths and angular resolution. When compared one-on-one with *Celonis*, *Dynamic Dagre* was able to keep up in overall quality. In three-way comparisons with *Celonis* and *Base Dagre*, it placed second, trading some quality for much higher stability.

7.2. Contributions

This thesis makes contributions in three main areas: algorithmic development, methodological guidance, and quantitative analysis.

Algorithm. The central contribution is the *Dynamic Dagre* algorithm, an enhanced version of the Dagre hierarchical layout algorithm. It extends the base algorithm with both general improvements and dynamic features:

- Even without enabling dynamic features, Dynamic Dagre differs from the original Dagre in several respects: source nodes are always placed at the top, antiparallel edges are grouped, and additional sweep iterations are performed to refine the global ordering. These changes generally reduce edge crossings and never worsen them compared to the baseline.
- With dynamic features enabled, the algorithm achieves high stability in relative ranks and orders by using information from the previous layout. This functionality is most effective when a graph is incrementally built up variant by variant, although even larger changes benefit from improved stability compared to static layouts.
- The *core path* is introduced as a novel concept that can be specified with or without dynamic features, though it is particularly powerful in combination with them. It ensures that a defined reference path is placed centrally and preserved with minimal changes, thereby providing an anchor around which the rest of the graph is arranged.

Dynamic Dagre is designed as a drop-in replacement for Dagre. In static settings, it is at least as good as the baseline, and in dynamic contexts it consistently provides superior stability while maintaining competitive quality. While the core path concept is motivated by process mining process models, it may prove useful in other domains as well. The algorithm is openly available on [GitHub](#).

Procedure model. This thesis also demonstrates how a systematic design science procedure model can guide the iterative improvement of an established algorithm. The approach illustrates how problems can be identified, conceptual solutions suggested, and modifications developed and evaluated in a modular, reproducible way. This process provides orientation for future extensions to Dagre or similar layout algorithms.

Analysis. Finally, the thesis contributes a quantitative analysis of layout stability and quality. The results highlight the strengths and limitations of Dynamic Dagre and present a methodology for evaluating dynamic layouts. This includes the use of comparative stability measures and quality assessment, which can serve as a basis for future studies on dynamic graph drawing.

7.3. Limitations

While *Dynamic Dagre* demonstrates clear advantages over the baseline Dagre algorithm, some limitations need to be acknowledged.

Graph constraints. As an extension of Dagre, the algorithm inherits the same requirements on the underlying graph. It can only produce hierarchical layouts, which presupposes a directed graph without parallel edges in the same direction. Graphs that undergo drastic structural changes between states, such as simultaneous addition and removal of multiple nodes and edges, may lead to less stable or visually suboptimal layouts. Although such extreme changes are unlikely in most process mining scenarios, their effects cannot be fully ruled out.

Evaluation scope. The algorithm was primarily tested on process mining graphs, with the BPI Challenge data set serving as the central benchmark. While some additional empirical checks on other graphs were performed, the evaluation was not extended to fundamentally different representations such as Petri nets. Similarly, the experiments focused on graphs of relatively small size. Larger models were only explored informally, where interpretability becomes challenging regardless of the layout algorithm. It remains possible that suboptimal trends in early stages of large dynamic graphs could accumulate and degrade later layouts.

Practical validation. The evaluation was carried out in a controlled research setting. Unfortunately, the algorithm could not be deployed in a production environment to obtain systematic feedback from real users and diverse customer data. Such field studies would provide a more nuanced understanding of its applicability, usability, and robustness in practice.

Comparative algorithms. There are relatively few layout algorithms that explicitly address dynamic stability, especially for hierarchical process models. Dedicated dynamic layout algorithms may in principle outperform Dynamic Dagre in stability, but no suitable candidates were identified during this work.

Metrics and measures. The evaluation employed established stability and quality metrics, but with limitations. Stability was assessed using one of nine measures proposed, since the others did not apply to this context. Nonetheless, the core principle of minimizing node movement is consistent with broader literature. Quality metrics were originally designed for force-directed layouts with simple point-like nodes, which may limit their applicability to hierarchical process models.

8. Conclusion and Future Work

Conclusion

Process mining enables the objective and holistic analysis of business processes. For the key tasks of exploration and conformance checking, process models are employed in the form of directed node–edge graphs. Hierarchical layouts are particularly well suited to visualize these models, as they respect the inherent direction and flow of processes and produce structured, interpretable diagrams.

In practical use, process models are often analyzed by incrementally adding process variants to a graph. This creates a sequence of related graph states that share many nodes and edges. For users to effectively interpret such evolving models, layouts must remain visually stable across these states, preserving mental maps while highlighting genuine changes. Achieving this goal is the domain of dynamic graph layout algorithms. However, despite the importance of this requirement, no established algorithm for dynamic layouts of process mining process models was available prior to this work.

This thesis introduced *Dynamic Dagre*, an extension of the established Dagre algorithm tailored for dynamic layouts. The algorithm was enhanced to prioritize stability, particularly in the relative ranking and ordering of nodes across versions. In addition, a novel concept of a *core path* was introduced: a designated path through the graph, often corresponding to the target process or most frequent variant, that remains centrally fixed and exceptionally stable, serving as a visual anchor for deviations.

Evaluation demonstrated that *Dynamic Dagre* significantly improves visual stability compared to the baseline Dagre implementation and significantly surpasses the commercial process mining tool Celonis in this respect. Visual quality improved in some aspects, such as edge crossings, while trade-offs emerged in others, notably uniform edge lengths. Overall, the algorithm represents a substantial advancement in dynamic layouts and is well suited for process model visualization in process mining contexts.

Future Work

Several avenues remain for further improvement and exploration:

Refinement of sweep heuristics. Dage's original implementation appears to intend the reuse of global orders across sweeps, but currently each sweep restarts from the initial order. Attempts to implement the intended strategy did not yield better results in this work, which may suggest limitations in the layer-by-layer heuristic itself. Further investigation could clarify whether this approach can be made effective or should be abandoned.

Adaptive biasing strategies. The current implementation favors one direction when integrating new elements, often placing them on the right. Exploring alternating or adaptive left-right biasing strategies may lead to more balanced layouts and reduced local crowding.

Broader empirical evaluation. Testing was largely restricted to process mining graphs, primarily from the BPI Challenge dataset. Applying *Dynamic Dage* to a wider range of datasets and use cases, including Petri nets or larger, more complex graphs, would provide deeper insights into its generalizability and scalability.

Appendices

```

1 function isReachedFromStartWithoutEdge(g, node, ignoreEdge) {
2   // reverse dfs from the given node to 'Start' without using edge e
3   let localVisited = {};
4
5   function reverseDfs(v) {
6     if (g.sources().includes(v)) {
7       return true;
8     }
9     localVisited[v] = true;
10    for (let e of g.inEdges(v)){
11      // do not use ignored edge
12      // do not visit the same node twice
13      if (!(e.v === ignoreEdge.v && e.w === ignoreEdge.w)
14        && !Object.hasOwn(localVisited, e.v)) {
15        if (reverseDfs(e.v)) {
16          return true;
17        }
18      }
19    }
20    return false;
21  }
22  return reverseDfs(node);
23 }

```

Figure 1.: Function to make sure an edge is safe to reverse

```

1 function compareByOldOrder(oldNodes, nodeA, nodeB) {
2   // early return 0 (equal) if any data is missing, or nodes are not
   edges
3   if (!(oldNodes && nodeA && nodeB &&
4     nodeA.dummy === "edge" && nodeB.dummy === "edge" &&
5     oldNodes[nodeA.edgeObj.v] && oldNodes[nodeB.edgeObj.v] && oldNodes[
      nodeA.edgeObj.w] && oldNodes[nodeB.edgeObj.w])) {
6     return 0;
7   }
8   // by default, we assume that we are looking in the forward direction (
      start -> end)
9   let isForward = true;
10  // if both edges have the same target, we are in the backwards
      direction (end -> start)
11  // since this comparison is only ever used on a rank immediately after
      a proper node from which multiple edges emerge
12  // that proper node is the common v (source) in the forward direction
      or the common w (target) in the backwards direction
13  if (nodeA.edgeObj.w === nodeB.edgeObj.w) {
14    isForward = false;
15  }
16  const targetRank = isForward? oldNodes[nodeA.edgeObj.v].rank + 1 :
      oldNodes[nodeA.edgeObj.w].rank - 1;
17
18  const oldA = Object.entries(oldNodes).find(n => n[1].edgeObj?.v ===
      nodeA.edgeObj.v && n[1].edgeObj?.w === nodeA.edgeObj.w && n[1].rank
      === targetRank);
19  const oldB = Object.entries(oldNodes).find(n => n[1].edgeObj?.v ===
      nodeB.edgeObj.v && n[1].edgeObj?.w === nodeB.edgeObj.w && n[1].rank
      === targetRank);
20  if (!oldA || !oldB) {
21    return 0;
22  }
23  const oldOrderA = oldA[1].order;
24  const oldOrderB = oldB[1].order;
25  return isNaN(oldOrderA - oldOrderB) ? 0 : oldOrderA - oldOrderB;
26 }

```

Figure 2.: Full function to compare the old order of two dummy nodes

A. Use of Generative Artificial Intelligence (GenAI) Tools

In the preparation of this Master's thesis, I made limited use of generative AI tools to support the writing and coding process. Their role was strictly assistive, and the intellectual and creative contributions of this work remain my own.

ChatGPT (OpenAI) and Claude (Anthropic) were used to review and improve the clarity and readability of certain formulations. These tools were not used to generate original content or arguments; all ideas, structure, and analysis originate from my own work.

GitHub Copilot was used as a coding assistant. In some cases, I accepted code suggestions for syntax, debugging, or efficiency improvements. All substantive implementations, algorithms, and design decisions are my own.

All AI-supported contributions were reviewed, adapted, and integrated critically by me. The substantial intellectual and creative achievements of this thesis are my own, in accordance with the declaration of authorship.

Bibliography

- [BK02] Ulrik Brandes and Boris Köpf. “Fast and Simple Horizontal Coordinate Assignment.” In: *Graph Drawing*. Ed. by Petra Mutzel, Michael Jünger, and Sebastian Leipert. Berlin, Heidelberg: Springer, 2002, pp. 31–44. ISBN: 978-3-540-45848-7. DOI: [10.1007/3-540-45848-4_3](https://doi.org/10.1007/3-540-45848-4_3).
- [BM12] Ulrik Brandes and Martin Mader. “A Quantitative Comparison of Stress-Minimization Approaches for Offline Dynamic Graph Drawing.” In: *Graph Drawing*. Ed. by Marc van Kreveld and Bettina Speckmann. Berlin, Heidelberg: Springer, 2012, pp. 99–110. ISBN: 978-3-642-25878-7. DOI: [10.1007/978-3-642-25878-7_11](https://doi.org/10.1007/978-3-642-25878-7_11).
- [BW10] Edward A Bender and S Gill Williamson. *Lists, Decisions and Graphs*. Google-Books-ID: vaXv_yhefG8C. S. Gill Williamson, 2010. 261 pp.
- [Cel] Celonis. *Process Intelligence, AI, and Process Mining | Celonis*. URL: <https://www.celonis.com/> (visited on 09/15/2025).
- [dag] dagre. *dagre*. GitHub. URL: <https://github.com/dagrejs/dagre/wiki/Home> (visited on 08/07/2025).
- [dyn] dynagraph. *dynagraph.org: Welcome*. URL: <https://www.dynagraph.org/> (visited on 09/15/2025).
- [ELS93] Peter Eades, Xuemin Lin, and W. F. Smyth. “A fast and effective heuristic for the feedback arc set problem.” In: *Information Processing Letters* 47.6 (Oct. 18, 1993), pp. 319–323. ISSN: 0020-0190. DOI: [10.1016/0020-0190\(93\)90079-0](https://doi.org/10.1016/0020-0190(93)90079-0).
- [Fen+12] Kun-Chuan Feng, Chaoli Wang, Han-Wei Shen, and Tong-Yee Lee. “Coherent Time-Varying Graph Drawing with Multifocus+Context Interaction.” In: *IEEE Transactions on Visualization and Computer Graphics* 18.8 (Aug. 2012). Conference Name: IEEE Transactions on Visualization and Computer Graphics, pp. 1330–1342. ISSN: 1941-0506. DOI: [10.1109/TVCG.2011.128](https://doi.org/10.1109/TVCG.2011.128).
- [Flu] Disco Fluxicon. *Process Mining and Automated Process Discovery Software for Professionals - Fluxicon Disco*. URL: <https://fluxicon.com/disco/> (visited on 09/15/2025).
- [FT08] Yaniv Frishman and Ayellet Tal. “Online Dynamic Graph Drawing.” In: *IEEE Transactions on Visualization and Computer Graphics* 14.4 (July 2008). Conference Name: IEEE Transactions on Visualization and Computer Graphics, pp. 727–740. ISSN: 1941-0506. DOI: [10.1109/TVCG.2008.11](https://doi.org/10.1109/TVCG.2008.11).

Bibliography

- [Gan+93] E.R. Gansner, E. Koutsofios, S.C. North, and K.-P. Vo. "A technique for drawing directed graphs." In: *IEEE Transactions on Software Engineering* 19.3 (Mar. 1993). Conference Name: IEEE Transactions on Software Engineering, pp. 214–230. ISSN: 1939-3520. DOI: [10.1109/32.221135](https://doi.org/10.1109/32.221135).
- [GBG12] Thomas E. Gorochowski, Mario di Bernardo, and Claire S. Grierson. "Using Aging to Visually Uncover Evolutionary Processes on Networks." In: *IEEE Transactions on Visualization and Computer Graphics* 18.8 (Aug. 2012), pp. 1343–1352. ISSN: 1941-0506. DOI: [10.1109/TVCG.2011.142](https://doi.org/10.1109/TVCG.2011.142).
- [Hay+13] Aki Hayashi, Tatsushi Matsubayashi, Takahide Hoshida, and Tadasu Uchiyama. "Initial Positioning Method for Online and Real-Time Dynamic Graph Drawing of Time Varying Data." In: *2013 17th International Conference on Information Visualisation*. 2013 17th International Conference on Information Visualisation. ISSN: 2375-0138. July 2013, pp. 435–444. DOI: [10.1109/IV.2013.57](https://doi.org/10.1109/IV.2013.57).
- [Her07] Abdi Hervé. "Z-scores." In: *Encyclopedia of measurement and statistics* 3 (2007), pp. 1055–1058.
- [Hua13] Weidong Huang. *An Aggregation-Based Overall Quality Measurement for Visualization*. June 11, 2013. DOI: [10.48550/arXiv.1306.2404](https://doi.org/10.48550/arXiv.1306.2404). arXiv: [1306.2404\[cs\]](https://arxiv.org/abs/1306.2404).
- [IEE] Task Force on Process Mining IEEE. *BPI Challenge 2019 - IEEE Task Force on Process Mining*. URL: <https://www.tf-pm.org/competitions-awards/bpi-challenge/2019> (visited on 09/15/2025).
- [JM97] Michael Jünger and Petra Mutzel. "2-Layer Straightline Crossing Minimization: Performance of Exact and Heuristic Algorithms." In: *Journal of Graph Algorithms and Applications* 1 (Jan. 1, 1997). ISSN: 978-981-02-4926-7. DOI: [10.1142/9789812777638_0001](https://doi.org/10.1142/9789812777638_0001).
- [joi] jointJS. *JavaScript diagramming library for interactive Uls – JointJS*. URL: <https://www.jointjs.com> (visited on 08/07/2025).
- [Kha11] Mumit Khan. "Longest path in a directed acyclic graph (DAG)." In: *CSE Algorithms, Brac Univ., Dhaka, Bangladesh* (2011).
- [Lez+15] Alfredo Ramos Lezama, Irene-Angelica Chounta, Tilman Göhnert, and H. Ulrich Hoppe. "Visual Stability in Dynamic Graph Drawings." In: *i-com* 14.3 (2015), pp. 220–230. DOI: [doi:10.1515/icom-2015-0038](https://doi.org/doi:10.1515/icom-2015-0038).
- [Mis+95] Kazuo Misue, Peter Eades, Wei Lai, and Kozo Sugiyama. "Layout Adjustment and the Mental Map." In: *Journal of Visual Languages & Computing* 6.2 (June 1, 1995), pp. 183–210. ISSN: 1045-926X. DOI: [10.1006/jvlc.1995.1010](https://doi.org/10.1006/jvlc.1995.1010).
- [MRA10] J. Mendling, H. A. Reijers, and W. M. P. van der Aalst. "Seven process modeling guidelines (7PMG)." In: *Information and Software Technology* 52.2 (Feb. 1, 2010), pp. 127–136. ISSN: 0950-5849. DOI: [10.1016/j.infsof.2009.08.004](https://doi.org/10.1016/j.infsof.2009.08.004).
- [Pro] ProM. *ProM Tools – ProM 6.14 has been released*. URL: <https://promtools.org/> (visited on 09/15/2025).

Bibliography

- [pro] process.science. *Process Mining tool for Power BI* | process.science. URL: <https://www.process-science.com/power-bi/product> (visited on 09/16/2025).
- [STT81] Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiro Toda. "Methods for Visual Understanding of Hierarchical System Structures." In: *IEEE Transactions on Systems, Man, and Cybernetics* 11.2 (Feb. 1981), pp. 109–125. ISSN: 2168-2909. DOI: [10.1109/TSMC.1981.4308636](https://doi.org/10.1109/TSMC.1981.4308636).
- [Van11] Wil M. P. Van Der Aalst. *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011. ISBN: 978-3-642-19344-6. DOI: [10.1007/978-3-642-19345-3](https://doi.org/10.1007/978-3-642-19345-3).
- [VK04] Vijay Vaishnavi and B Kuechler. "Design Science Research in Information Systems." In: *Association for Information Systems* (Jan. 1, 2004).
- [yWo] yWorks. *yFiles for HTML Demo Applications - Interactive Hierarchical Layout*. yWorks, the diagramming experts. URL: <https://live.yworks.com/demos/layout/interactive-hierarchical/> (visited on 09/15/2025).
- [ZC06] Ping Zhang and Gary Chartrand. "Introduction to graph theory." In: *Tata McGraw-Hill New York* (2006).
- [Zho+13] Hong Zhou, Panpan Xu, Xiaoru Yuan, and Huamin Qu. "Edge bundling in information visualization." In: *Tsinghua Science and Technology* 18.2 (Apr. 2013), pp. 145–156. ISSN: 1007-0214. DOI: [10.1109/TST.2013.6509098](https://doi.org/10.1109/TST.2013.6509098).

Eidesstattliche Erklärung

Hiermit versichere ich an Eides statt, dass ich die vorliegende Arbeit im Masterstudiengang Informatik selbstständig verfasst und keine anderen als die angegebenen Hilfsmittel – insbesondere keine im Quellenverzeichnis nicht benannten Internet-Quellen – benutzt habe. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen wurden, sind als solche kenntlich gemacht. Ich versichere weiterhin, dass ich die Arbeit vorher nicht in einem anderen Prüfungsverfahren eingereicht habe. Sofern im Zuge der Erstellung der vorliegenden Abschlussarbeit generative Künstliche Intelligenz (gKI) basierte elektronische Hilfsmittel verwendet wurden, versichere ich, dass meine eigene Leistung im Vordergrund stand und dass eine vollständige Dokumentation aller verwendeten Hilfsmittel gemäß der Guten Wissenschaftlichen Praxis vorliegt. Ich trage die Verantwortung für eventuell durch die gKI generierte fehlerhafte oder verzerrte Inhalte, fehlerhafte Referenzen, Verstöße gegen das Datenschutz- und Urheberrecht oder Plagiate.

Hamburg, October 3, 2025

Mats Schrader
Mats Schrader

Veröffentlichung

Ich stimme der Einstellung der Arbeit in die Bibliothek des Fachbereichs Informatik zu.

Hamburg, October 3, 2025

Mats Schrader
Mats Schrader